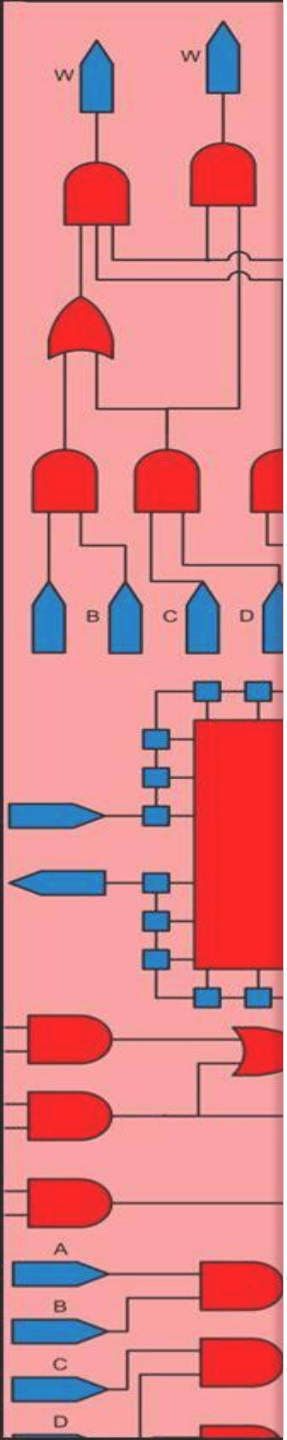


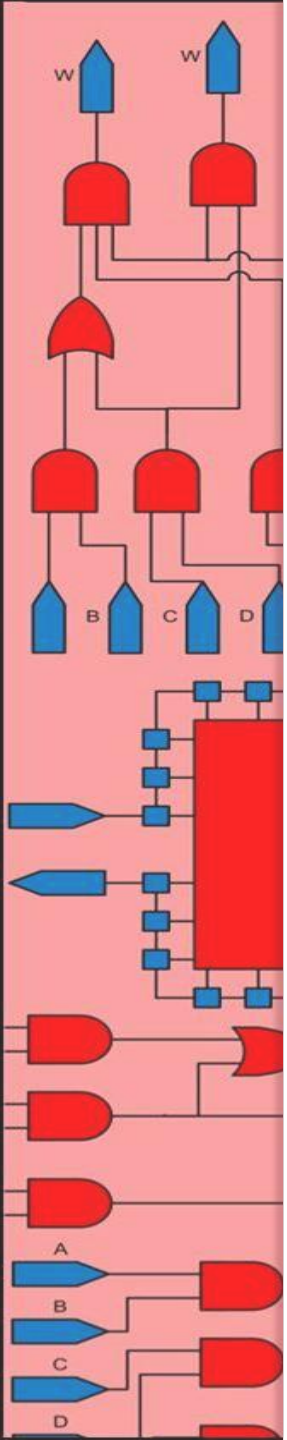
Topic 7

Verification



Outline

- Introduction to Digital Verification
- Simulation-based Verification
- SystemVerilog Constructs for Testbenches
- Constrained Random Test Generation
- SystemVerilog Assertions (SVA)
- SVA Case Study

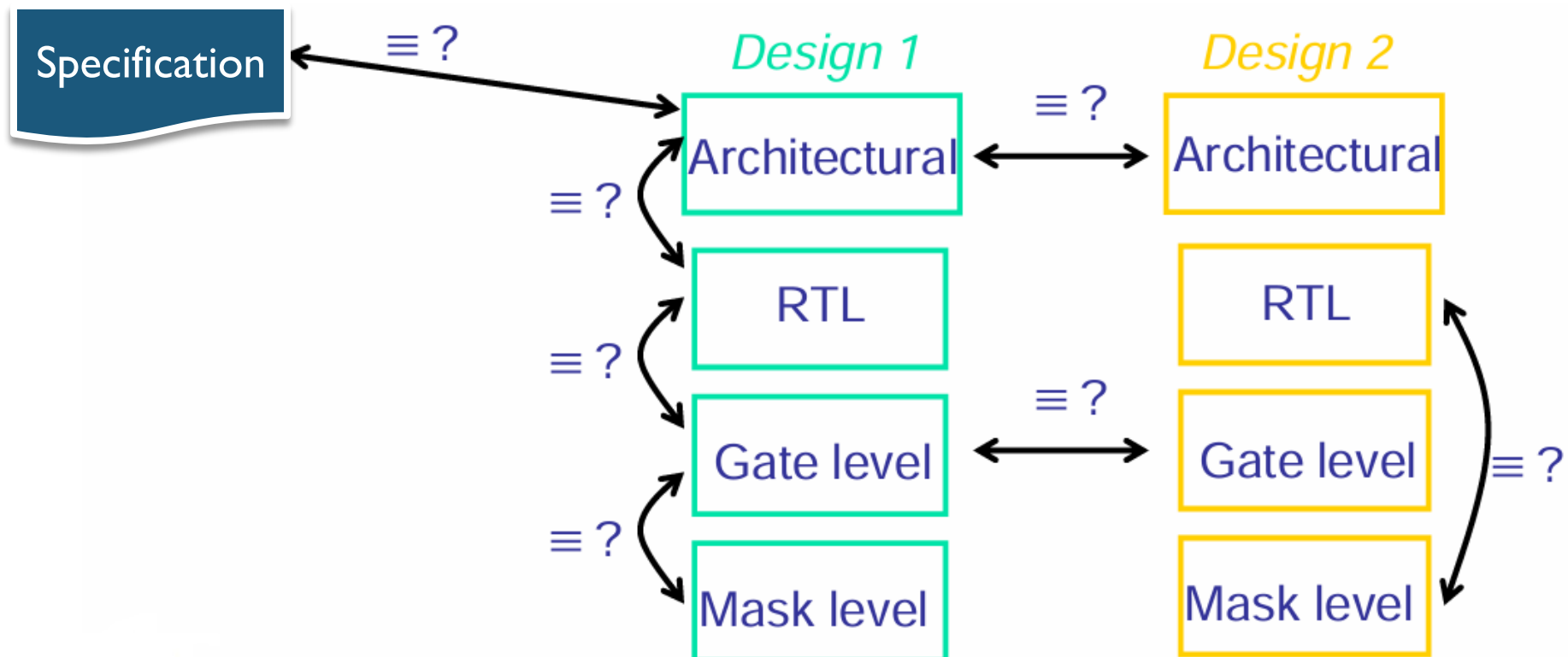


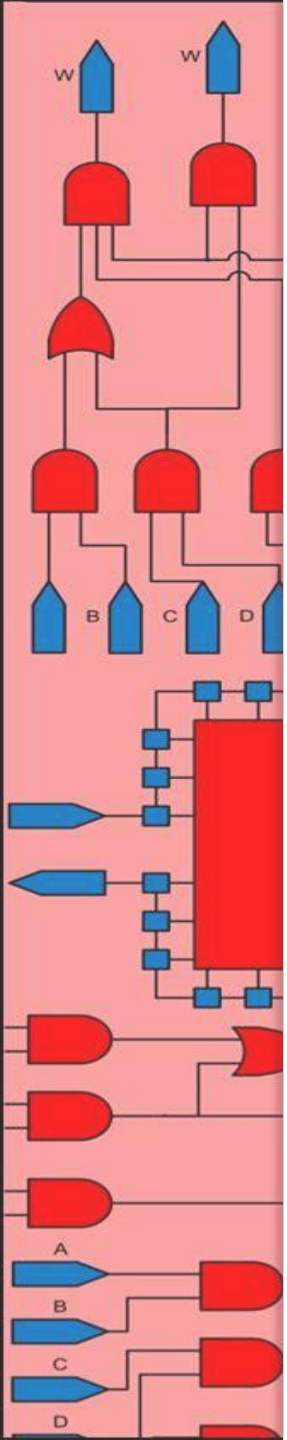
Outline

- **Introduction to Digital Verification**
- Simulation-based Verification
- SystemVerilog Constructs for Testbenches
- Constrained Random Test Generation
- SystemVerilog Assertions (SVA)
- SVA Case Study

What is Verification?

- Design verification = ensuring the correctness of the design
 - against its implementation (at different levels)
 - against alternative design (at the same level)

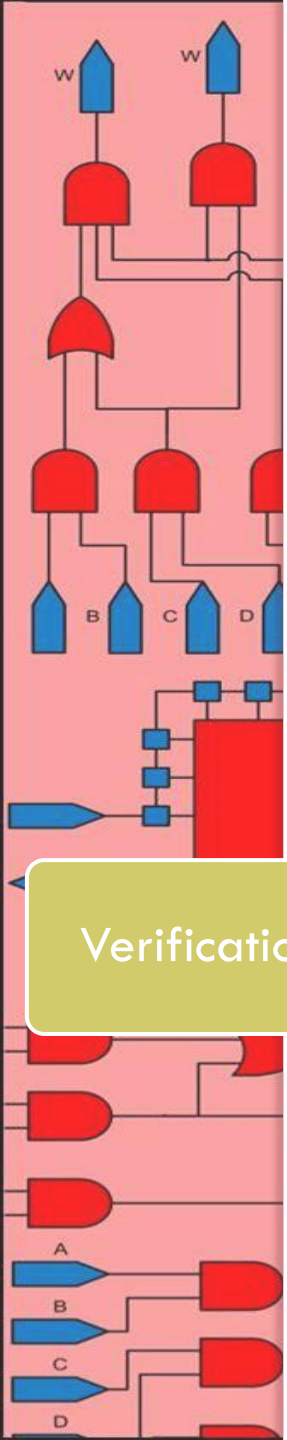




Why & Where Verification?

- Handles Design Complexity - Necessitates advanced and systematic verification methodologies
- Ensures functional correctness – Confirms that the circuit performs as specified
- Detects design bugs early – Saves cost and time by avoiding late-stage failures
- Applies Throughout the Design Flow – From RTL to transistor-level
- Local Optimizations

Verification Methods



Verification

Functional

Performance

Formal
Techniques

Model
checking (MC)

Equivalence
checking (EC)

Explicit MC
+Partial Order
Reduction (POR)

Symbolic MC
(SMC)

Bounded MC
(BMC)

Simulation/
Emulation-based
Techniques

SW-based

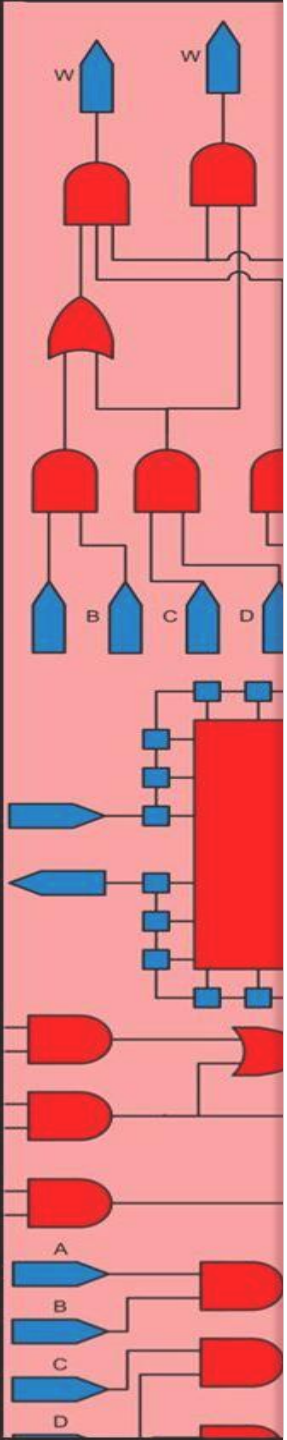
Universal Verification
Methodology (UVM)

Assertion-Based
Verification (ABV)

Hybrid
Techniques

HW-based

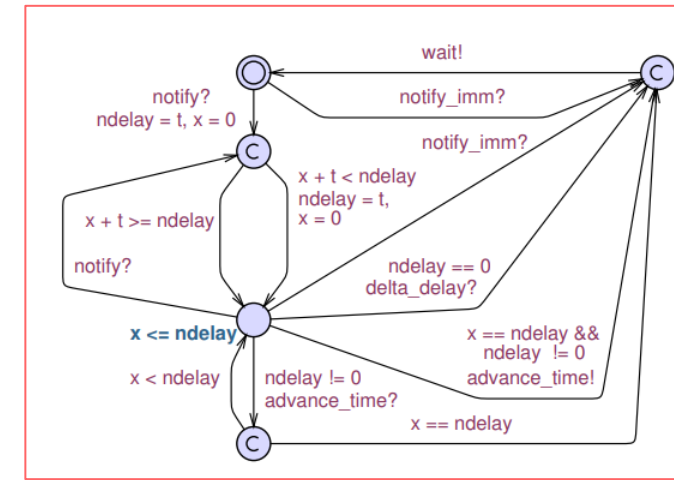
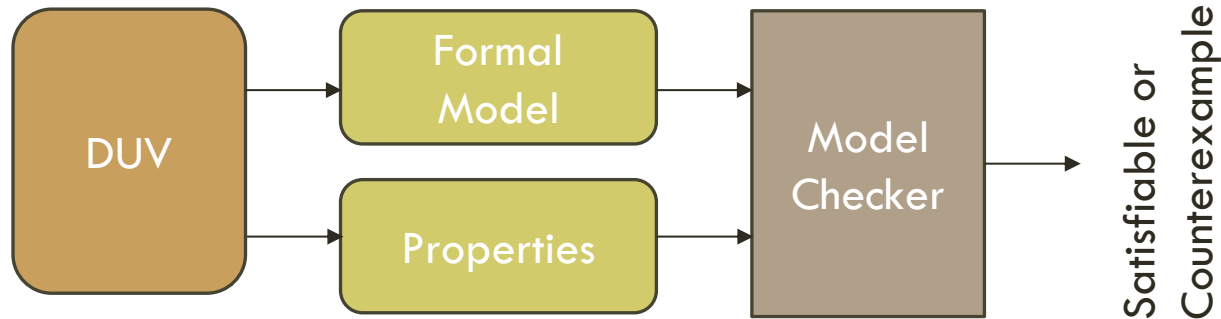
Hardware-in-Loop
(HIL)



Verification Methods

- **Formal methods** use mathematical techniques to prove the correctness of a design
 - Targeting **corner cases** or rare bugs: exploring all possible states
 - Design is **relatively small**: due to the state-space explosion problem
 - **Specifications** are well-defined: For formal to be effective, you need strong properties
- **UVM** is a simulation-based, coverage-driven verification framework
 - **Complex** designs: Reusable and scalable UVM environments
 - Functional testing with constrained-random or directed inputs
 - Comprehensive test plans: If you need to achieve functional coverage metrics
- **Assertions** embedded in the design or testbench, are checks used alongside simulations
 - **Complementing** UVM or other simulation techniques
 - Focus is on protocol or interface checking: handshake signals like *valid* and *ready* follow the protocol
 - Quick debugging: Assertions point directly to violations, aiding in rapid verification

Model Checking

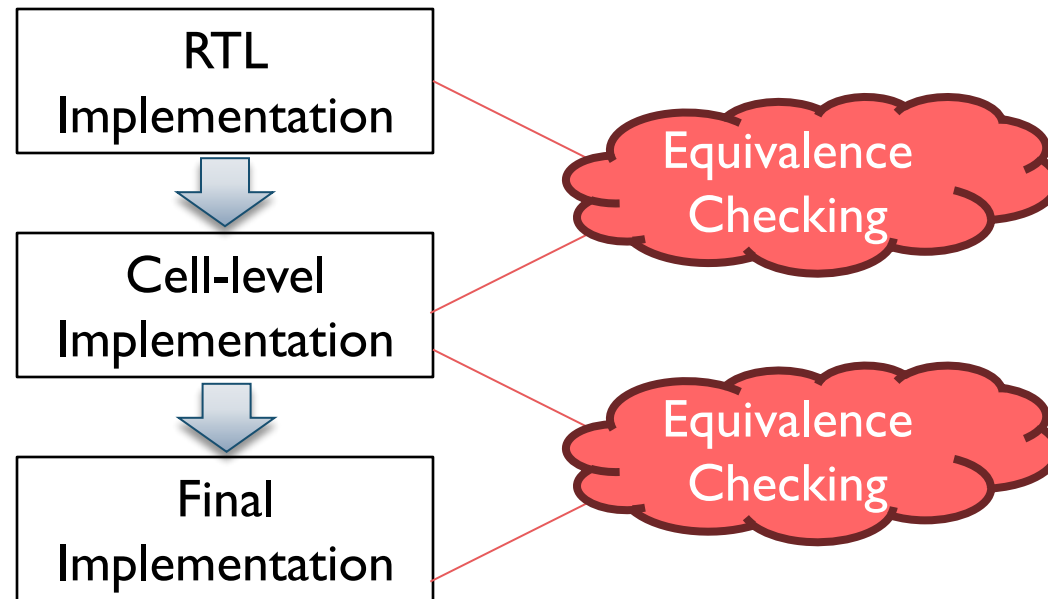


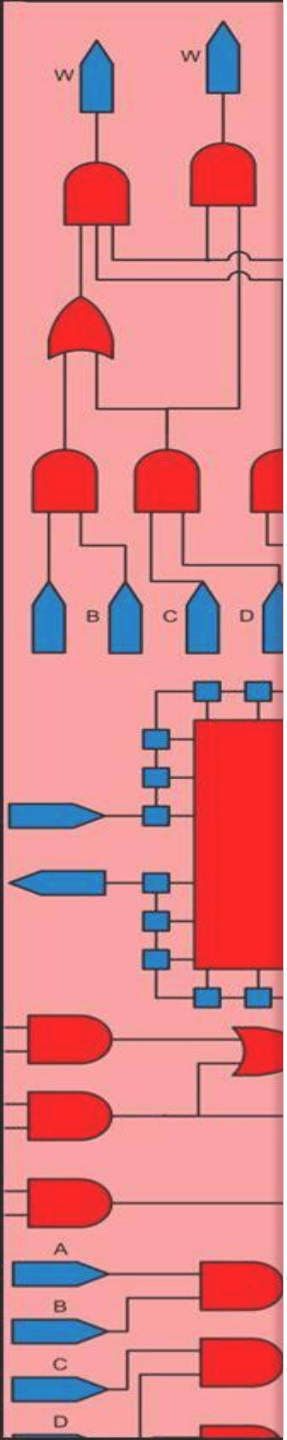
Timed Automata model

- **Formal model** of DUV in the **form of states** - **Automatically**
- **Properties** of DUV in the **form of temporal logic**, like Assertions, LTL, or CTL - **Manually**
- **Model checker** tool to formally prove whether these properties hold under all circumstances (Graph Algorithm, BDDs, SAT solvers)
- **Main Problems:** Circuit size & State-space explosion

Equivalence Checking

- Checks if two circuits are equivalent





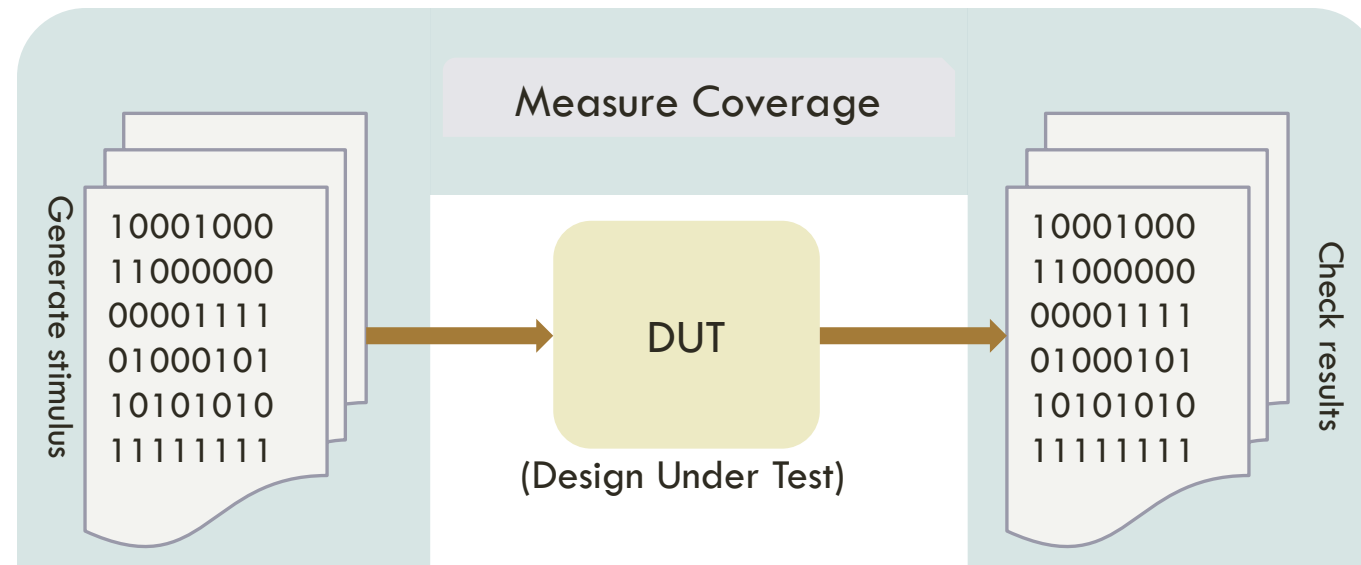
Outline

- Introduction to Digital Verification
- **Simulation-based Verification**
- SystemVerilog Constructs for Testbenches
- Constrained Random Test Generation
- SystemVerilog Assertions (SVA)
- SVA Case Study

Simulation-based Techniques

An Abstract Model
(Functional, Transaction
Level Model (TLM))

- Generating Stimulus
- Checking results vs expected outputs or Golden Model
- Measuring Coverage

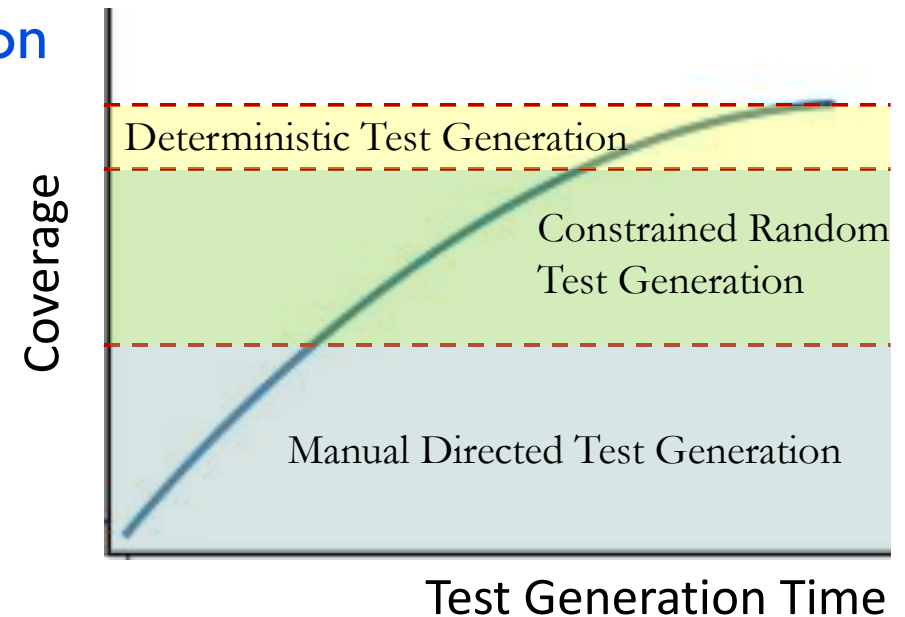


Simulation-based Techniques

Stimulus Generation

- Manual Directed Test Generation
 - ✓ Easy to generate, Predictable coverage
 - ✗ Not efficient (covers a small portion of the design)
- Constrained Random Test Generation
 - ✓ Fast to generate
 - ✗ Unpredictable coverage
- Deterministic Test Generation
 - ✓ High coverage
 - ✗ Hard to generate

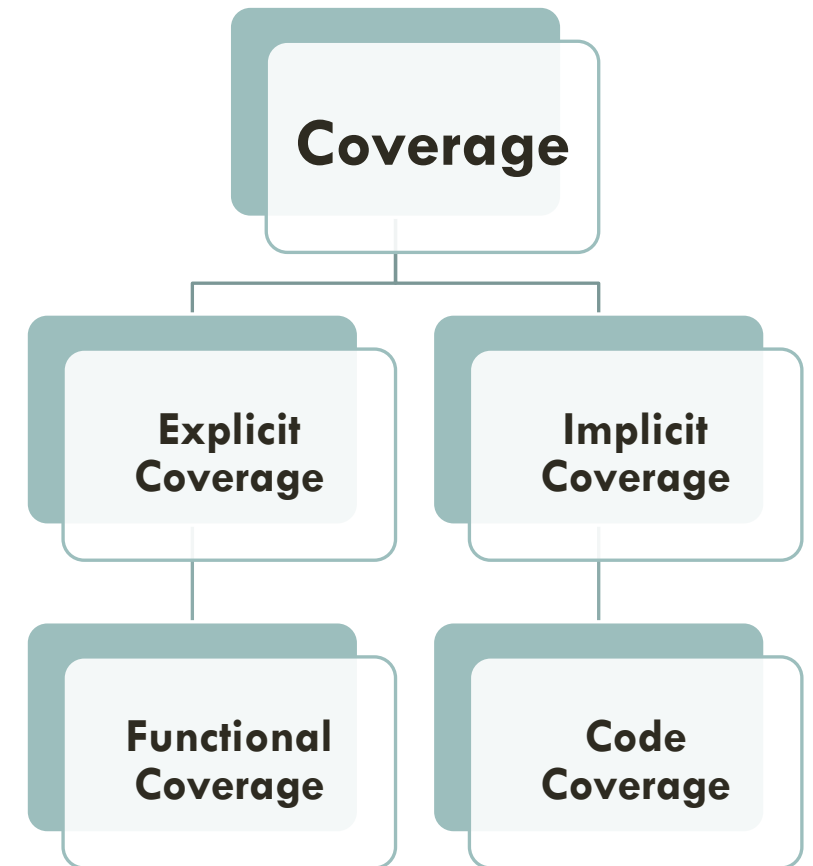
Exhaustive testing is not possible!



Simulation-based Techniques

Coverage Measurement

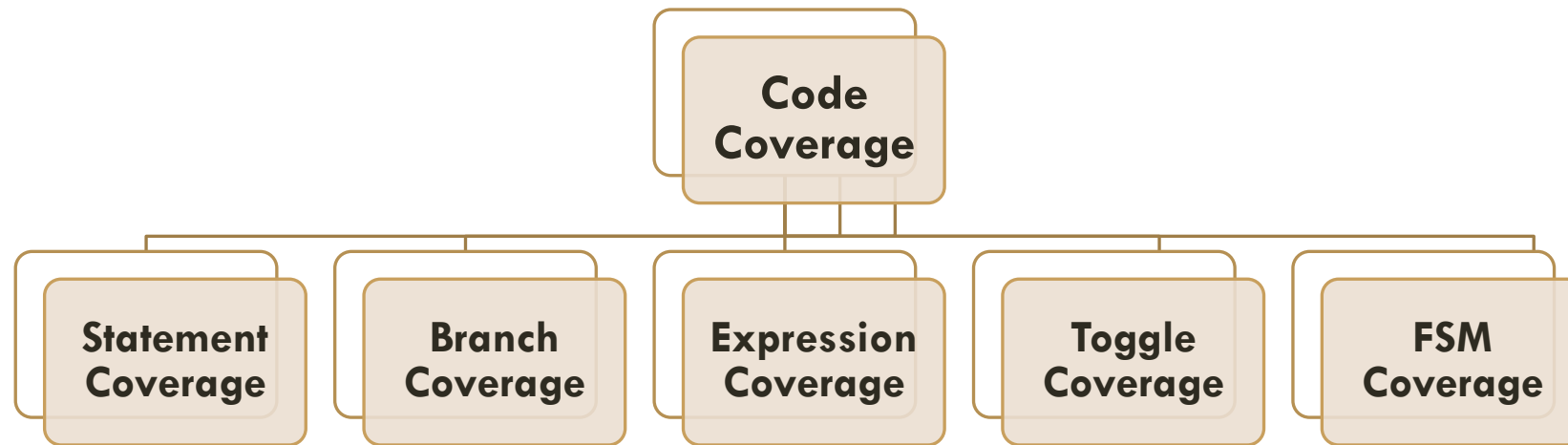
- Measuring the completeness of testing by monitoring which specific functionalities, properties, or events within a design have been exercised during simulation
- **Implicit Coverage:** Automatically generated by tools
- **Explicit Coverage:** A manually defined coverage model by the verification engineer



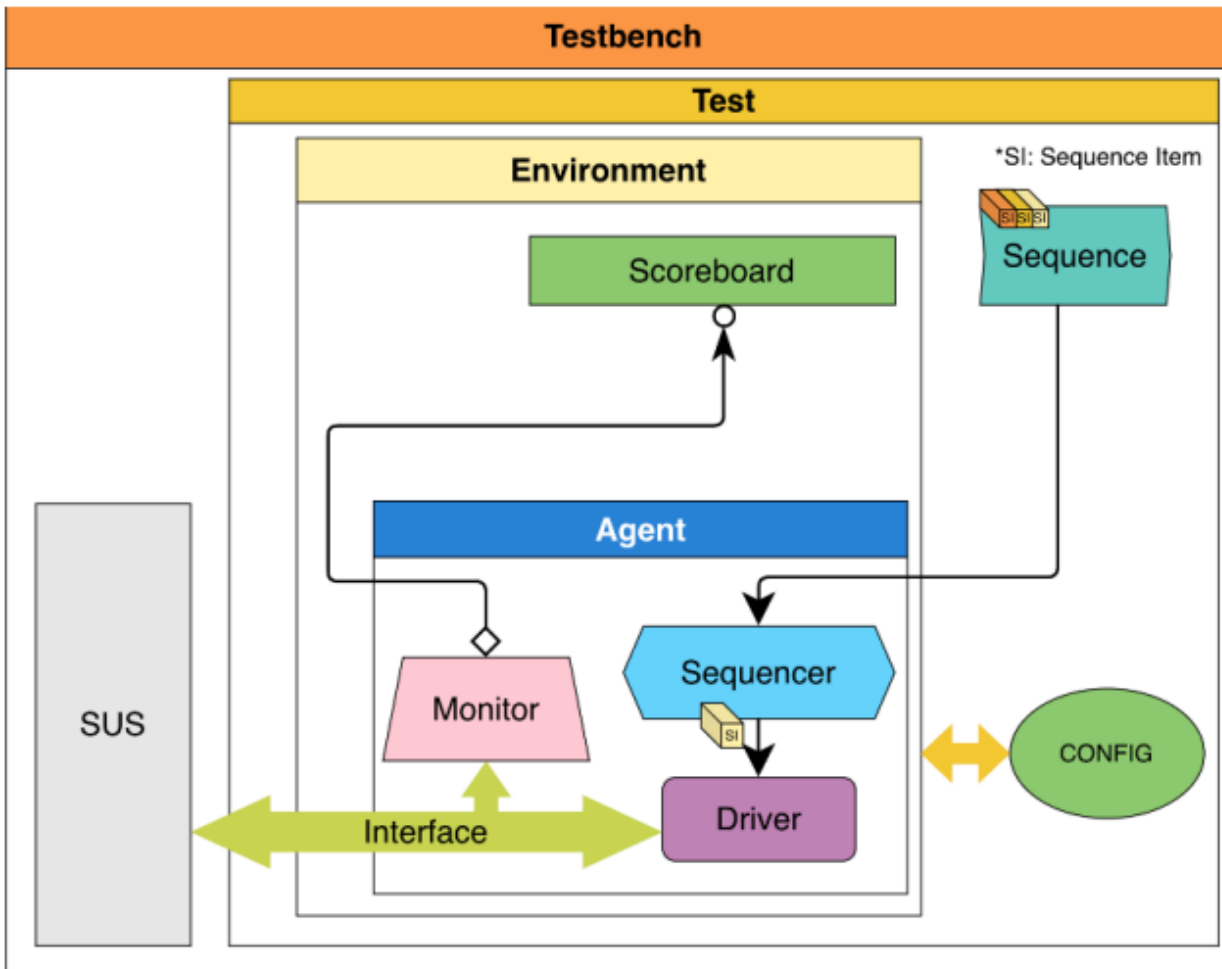
Simulation-based Techniques

Code Coverage

- **Statement Coverage:** Checks if each statement in the design was executed at least once.
- **Branch Coverage:** measures whether each possible branch in the control flow of the design has been executed during simulation.
- **Expression/Condition Coverage:** Verifies that each boolean condition in expressions has been evaluated to both true and false.
- **Toggle Coverage:** Measures how often individual bits of signals toggle ($0 \rightarrow 1$ and $1 \rightarrow 0$) during simulation
- **FSM Coverage:** Verifies that all states and transitions in an FSM are exercised during simulation.



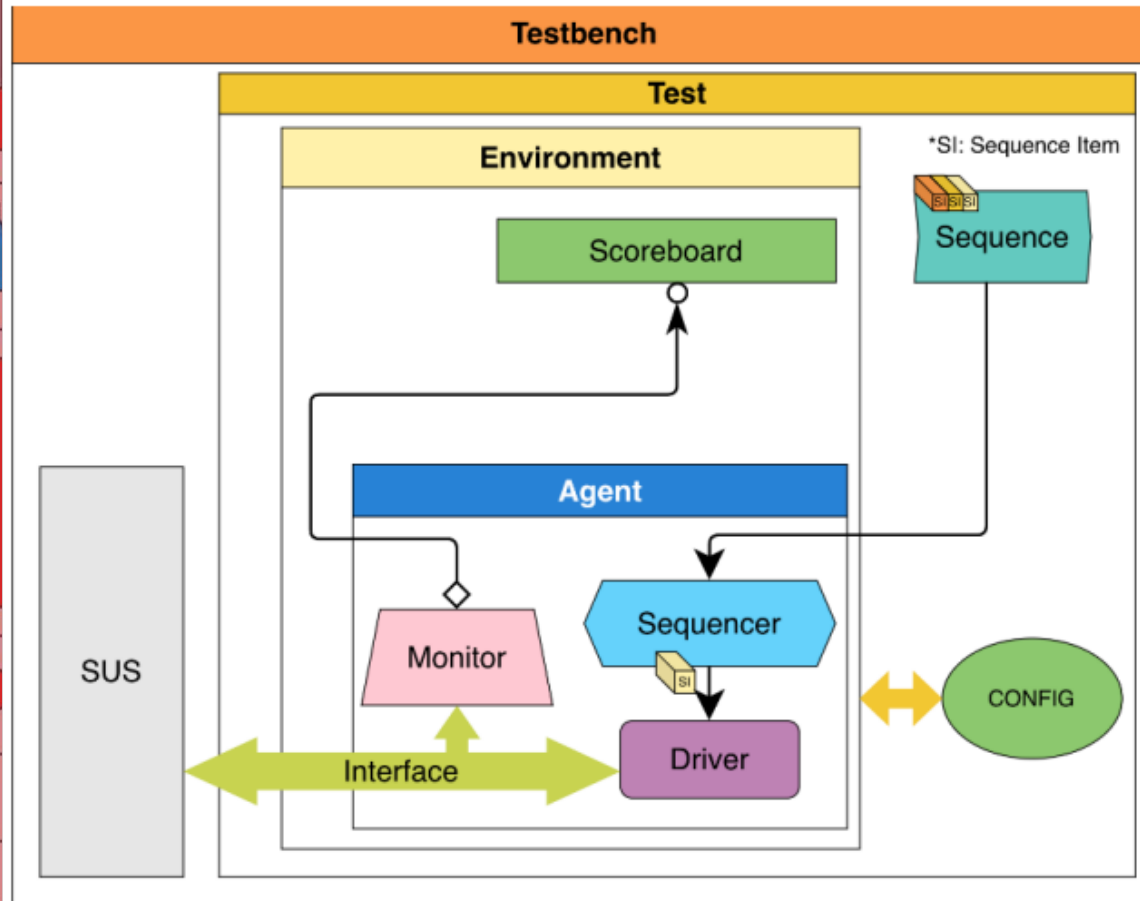
Universal Verification Methodology (UVM)



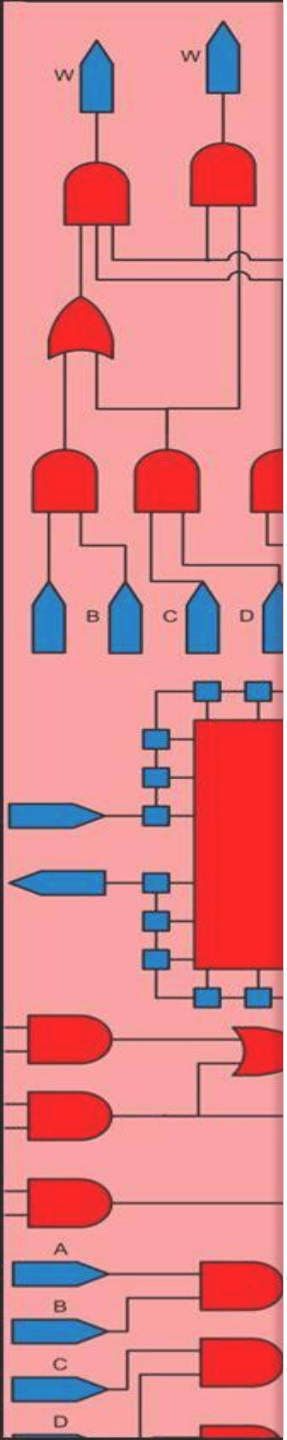
- Systematic testbench architectures
 - Modularity
 - Scalability
 - Configurability
 - Reusability

Language-agnostic
SV, e, SC

Universal Verification Methodology (UVM)

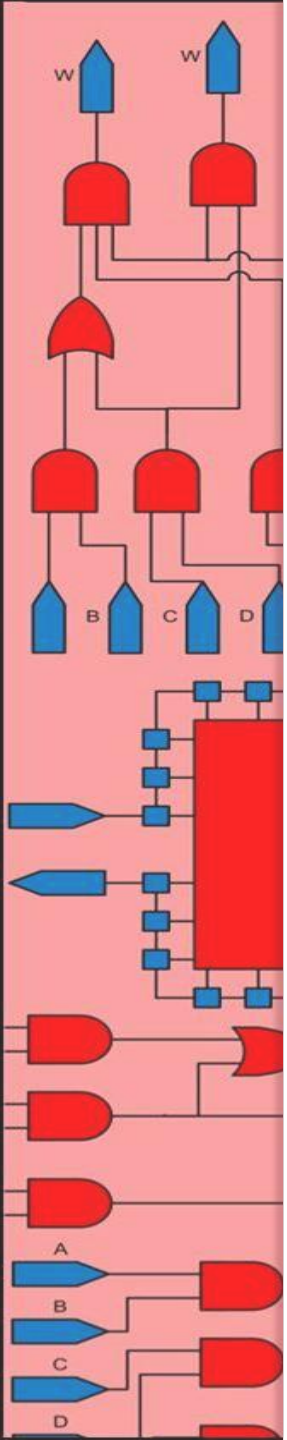


- Sequence generates sequence items
- Sequencer controls the flow of sequence items
- Driver drives the sequence items to the inputs of the DUT
- Monitor observes and keeps track of the DUT's output
- Scoreboard checks the behavior of the DUT and reports errors
- Agent & Environment are hierarchical components that group other verification components
- Test is the top-level UVM component



Outline

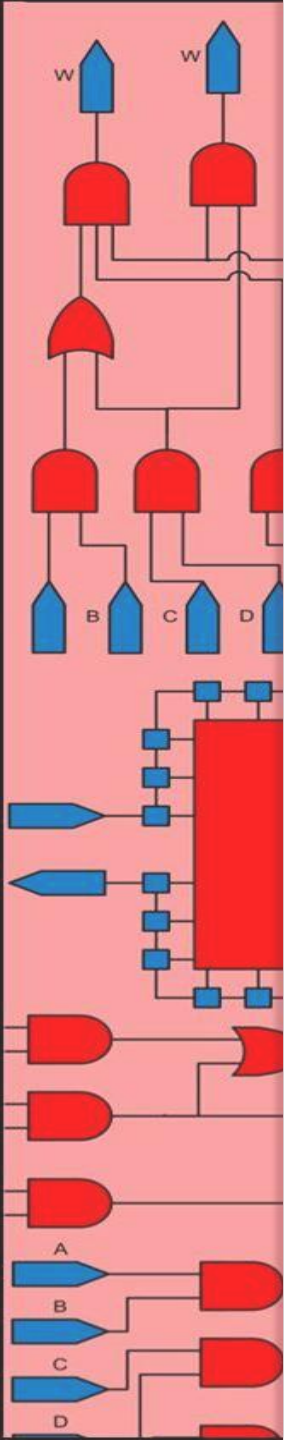
- Introduction to Digital Verification
- Simulation-based Verification
- **SystemVerilog Constructs for Testbenches**
- Constrained Random Test Generation
- SystemVerilog Assertions (SVA)
- SVA Case Study



SystemVerilog Constructs for Testbench

- Task
- Function
- Program Block
- Clocking Block
- Classes
- ...

Reference: IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800™



SystemVerilog Constructs for Testbench

- Task:
 - Is a callable subroutine without a return value
 - Can include time concepts (@, #, wait, ...)
 - Declared inside modules or programs is *automatic* by default, meaning each call has its own locals
 - **Use Cases:** For complex or reusable behaviors that may involve delays or multiple outputs, often used in testbenches
 - Synthesizable if executed in 0 time (contains only **blocking assignments**); otherwise, it is not synthesizable

A task with no timing is effectively a *function*

Syntax:

```
task [automatic] task_name (input type arg1, output type arg2);  
    // procedural statements  
    // can use delay and posedge  
endtask
```

SystemVerilog Constructs for Testbench

- Task:
 - Simulation Example

Task Usage

```
task automatic call_cells ();
begin
    wait_for_processor_ready();
    @(negedge clk);
    mem_valid = 1'b1;
    mem_addr  = MMIO_BASE_ADDR + MEM_ADDR_CALL_OFFSET;
    mem_wdata = 32'h0000_0001;
    mem_wstrb = 4'b0001;
    @(negedge clk);
    @(negedge clk);
    mem_valid = 1'b0;

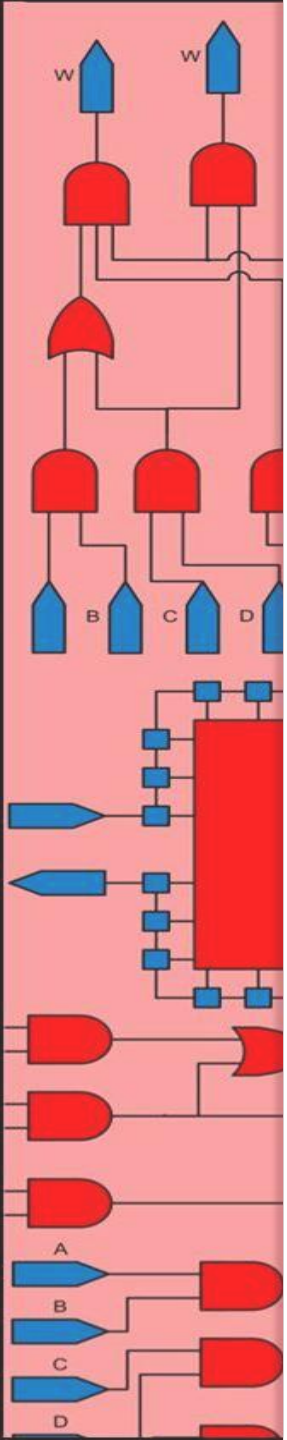
    wait_for_processor_ready();
    @(negedge clk);
    mem_valid = 1'b1;
    mem_addr  = MMIO_BASE_ADDR + MEM_ADDR_CALL_OFFSET;
    mem_wdata = 32'h0000_0000;
    mem_wstrb = 4'b0001;
    @(negedge clk);
    @(negedge clk);
    mem_valid = 1'b0;
end
endtask
```

Task Declaration

```
...
$display("Calling cells at time %t", $time);
call_cells();

// Wait for all instructions to be processed by DRRA
mem_valid = 1'b0;
#600;

// Meanwhile, do some scalar operations
$display("Reading the input buffer at time %t", $time);
reading_IB(
    24'h00_000b, // ib_saddr
    7'h01,       // ib_stride
    16'h0004,    // dm_saddr
    7'h02,       // dm_stride
    8'h06        // agu_counts
);
...
```



SystemVerilog Constructs for Testbench

- **Function:**
 - Returns a value
 - Cannot consume simulation time (Cannot include time concepts like @, #, wait)
 - Declared inside modules or programs is *automatic* by default, meaning each call has its own locals
 - **Use Cases:** For complex or reusable behaviors; intended for describing a combinational logic
 - Are widely used in synthesizable RTL as pure combinational

Syntax:

```
function [automatic] return_type func_name (input type arg);  
    // computations  
    return value;  
endfunction
```


SystemVerilog Constructs for Testbench

- Function:
- Simulation Example

Function Declaration

```
function automatic logic [PICO_DATA_WIDTH-1:0] apply_write_strobe (  
    input logic [PICO_DATA_WIDTH-1:0] mem_wdata,  
    input logic [PICO_DATA_WIDTH/8-1:0] mem_wstrb  
);  
    logic [PICO_DATA_WIDTH-1:0] result;  
    result = 0;  
    for (int i = 0; i < PICO_DATA_WIDTH/8; i++) begin  
        if (mem_wstrb[i])  
            result[i*8 +: 8] = mem_wdata[i*8 +: 8];  
    end  
  
    return result;  
endfunction
```

Function Usage

```
always_comb begin : DECODER_WRITE  
    // Set all outputs to inactive values  
    if (mem_valid) begin  
        if ((mem_addr >= (MMIO_BASE_ADDR + MMDM_BASE_ADDR)) &&  
            (mem_addr < (MMIO_BASE_ADDR + MMDM_BASE_ADDR + MMDM_LENGTH))) begin  
            dm_enable = 1'b1;  
            dm_addr = (mem_addr[DM_ADDR_WIDTH-1:0] - (MMIO_BASE_ADDR + MMDM_BASE_ADDR)) >> 2;  
            dm_wdata = apply_write_strobe (mem_wdata, mem_wstrb);  
        end else if ((mem_addr >= MMIO_BASE_ADDR) &&  
            (mem_addr < (MMIO_BASE_ADDR + MMIO_LENGTH))) begin  
            case (mem_addr[7:4])  
                4'h0 : begin  
                    case (mem_addr[3:0])  
                        4'h0 : begin  
                            reg_config_we = 1'b1;  
                            reg_config_di = apply_write_strobe (mem_wdata, mem_wstrb);  
                        end  
                        4'h4 : begin  
                            reg_instrsaddr_we = 1'b1;  
                            reg_instrsaddr_di = apply_write_strobe (mem_wdata, mem_wstrb);  
                        end  
                        4'h8 : begin  
                            reg_instrnums_we = 1'b1;  
                            reg_instrnums_di = apply_write_strobe (mem_wdata, mem_wstrb);  
                        end  
                        4'hc : begin  
                            reg_instrdata_we = 1'b1;  
                            reg_instrdata_di = apply_write_strobe (mem_wdata, mem_wstrb);  
                        end  
                    end  
                end  
            endcase  
        end  
    end  
end
```


SystemVerilog Constructs for Testbench

Races Between the DUT & the TB



Why does it happen?

- Occurs when two or more processes try to read or write the same signal at the same simulation time, and the order of execution is undefined
- The result depends on simulator scheduling, not design intent
- Leads to unpredictable behavior during simulation

```
module dut(input logic clk, input logic d, output logic q);  
    always_ff @(posedge clk)  
        q <= d;  
endmodule
```

```
module tb;  
    logic clk = 0, d;  
    logic q;  
  
    dut u1 (.clk(clk), .d(d), .q(q));  
  
    always #5 clk = ~clk;  
  
    initial begin  
        d = 0;  
        #9 d = 1;  
        #10 d = 0;  
        #10 $finish;  
    end  
endmodule
```

SystemVerilog Constructs for Testbench

- Program:
 - Encapsulates testbench code and ensures safe interaction with the DUT
 - Declared at the top level (like a module)
 - Only for testbenches and verification, **not synthesizable**
 - Can contain *initial/always* blocks, *tasks*, *functions*, and other testbench elements
 - **Use case:** race-free interaction between the DUT and the TB.
 - Executed in a different simulation region than the modules, so that stimulus and checking happen cleanly after design logic settles

solution!

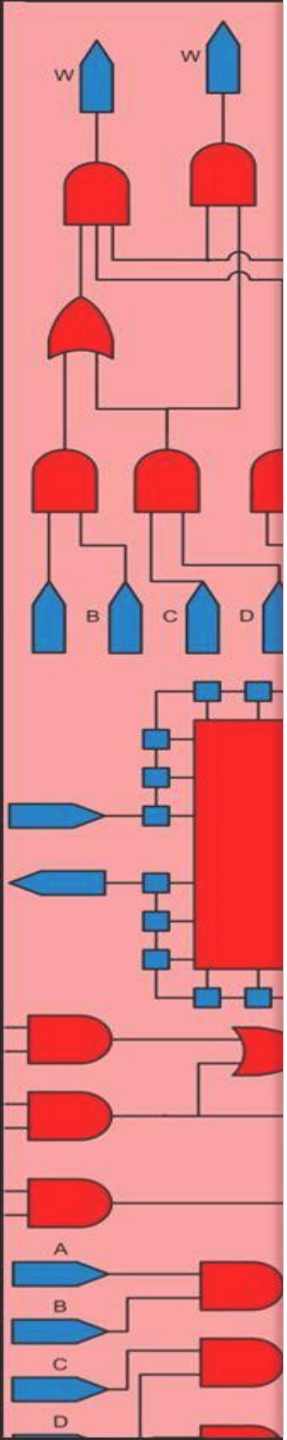


Syntax:

```
program [automatic] program_name (input type arg1, output type arg2);  
    // declarations  
    initial begin  
        // stimulus code  
    end  
endprogram
```

SystemVerilog Constructs for Testbench

- Program:
 - Simulation Example



```
module counter (  
    input logic clk,  
    input logic rst,  
    output logic [3:0] count  
);  
    always_ff @(posedge clk or posedge rst)  
        if (rst)  
            count <= 0;  
        else  
            count <= count + 1;  
endmodule
```

```
program automatic test (  
    input logic clk,  
    output logic rst,  
    input logic [3:0] count  
);  
    initial begin  
        rst = 1;  
        repeat (2) @(posedge clk);  
        rst = 0;  
  
        repeat (5) @(posedge clk);  
        $display("Time=%0t, Count=%0d", $time, count);  
  
        $stop;  
    end  
endprogram
```

SystemVerilog Constructs for Testbench

- Program:
 - Simulation Example

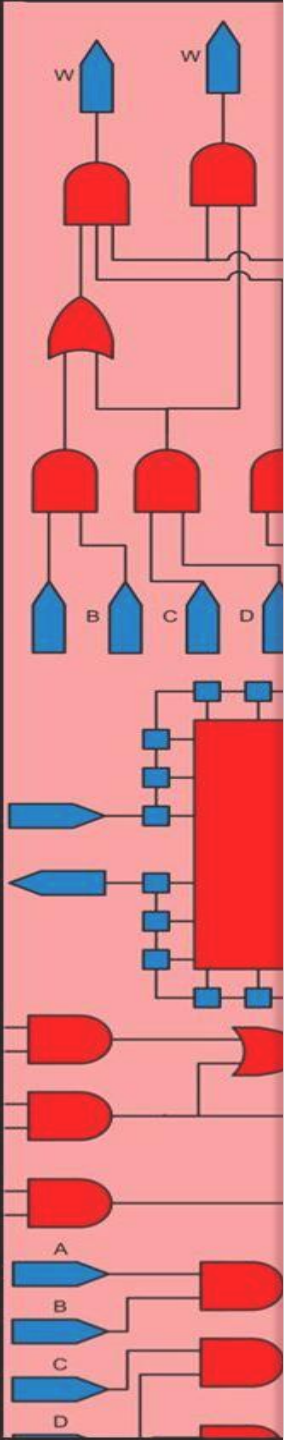
Don't generate a Clock inside a *program* Block

- Active region: DUT looks for posedge clk
- Reactive region: The program runs after that edge is already processed.

```
module top;  
  logic clk = 1'b0;  
  logic rst = 1'b0;  
  logic [3:0] count;  
  
  always #5 clk = ~clk;  
  
  counter dut (  
    .clk(clk),  
    .rst(rst),  
    .count(count)  
  );  
  
  test tb (  
    .clk(clk),  
    .rst(rst),  
    .count(count)  
  );  
endmodule
```

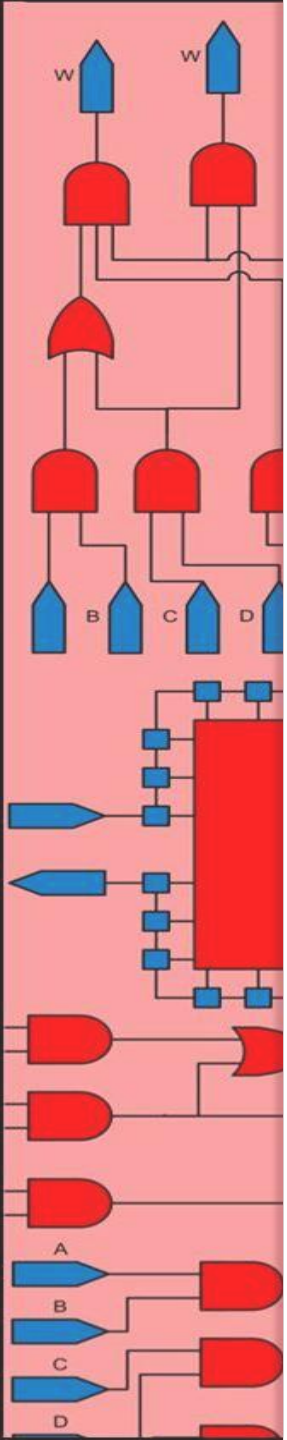
Instantiate DUT

Instantiate Program Block



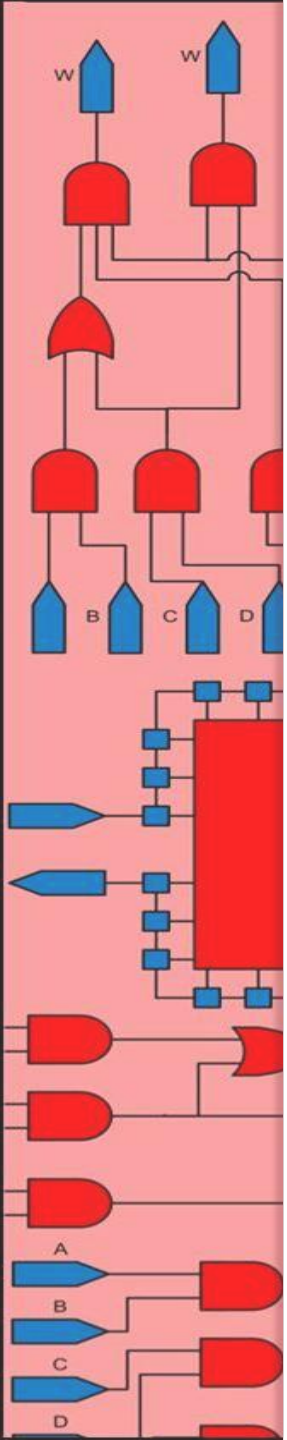
Outline

- Introduction to Digital Verification
- Simulation-based Verification
- SystemVerilog Constructs for Testbenches
- **Constrained Random Test Generation**
- SystemVerilog Assertions (SVA)
- SVA Case Study



Constrained Random Test Generation

- A powerful methodology for generating realistic and diverse test scenarios
- Used to define the valid range of values for input ports and ensure that the generated values meet specific criteria
- Constraints processed by an internal constraint solver that tries to find random values satisfying all constraints
- If constraints contradict each other, randomization fails



Constrained Random Test Generation

Core Concepts

SystemVerilog is an
object-oriented language

- **class variable** — Defines the structure and behavior of your test object
 - Define random variables and constraints inside a class
 - Then, create objects from it for randomization
- **Random variables** — Make it randomizable
 - *rand* → Random variable (can repeat values)
 - *randc* → Random cyclic variable (will iterate through all possible values before repeating)
- **Constraint Blocks** — Control how to randomize
 - Define rules using the *constraint* block
- **Randomization** — Executes it
 - Call *object.randomize()* to generate random values that obey the constraints

Constrained Random Test Generation

- Example 1

```
class Packet;  
  rand logic [7:0] addr;  
  rand logic [7:0] data;  
  rand logic write_enable;  
  
  constraint c_addr_byte_access {addr[1:0] == 2'b0;}  
  constraint c_write_enable {  
    if (write_enable) addr % 2 == 0;  
  }  
  
  function void display();  
    $display("ADDR = %0h, DATA = %0h, WRITE_EN = %0b",  
            addr, data, write_enable);  
  endfunction  
endclass
```

Class Variable

Declaration

Random Variables

Constraints

You cannot use X or Z as actual values inside randomization constraints
constraint c1 { a == 1'bx; } ❌ not allowed

Usage

```
module tb;  
  Packet pkt;  
  
  initial begin  
    pkt = new();  
  
    repeat (10) begin  
      if (pkt.randomize()) pkt.display();  
      else $display("Randomization failed!");  
    end  
  end  
endmodule
```

Randomization

Constrained Random Test Generation

Basic Concepts

- Random Sequence

- A **transaction** is one random data item (like a packet or bus operation)
- A **sequence** is a collection of such items, generated one after another (series of transactions)

Specify per-call rules

- Inline random variables

- Are **temporary variables** that you declare inside a *randomize()* call — instead of being part of the class
- Designate the complete set of random variables within that object; all other variables in the object are considered state variables regardless of the class definition

- Inline constraints

- Are **temporary constraints** used with the *with* operator
- Allow you to override or add constraints at runtime without modifying your class

Constrained Random Test Generation

- Example 2

Usage

Declaration

```
class Packet;
  rand logic [7:0] addr;
  rand logic [7:0] data;
  rand logic write_enable;

  constraint c_addr_byte_access {addr[1:0] == 2'b0;}
  constraint c_write_enable {
    if (write_enable) addr % 2 == 0;
  }

  function void display();
    $display("ADDR = %0h, DATA = %0h, WRITE_EN = %0b"
             addr, data, write_enable);
  endfunction
endclass
```

Inline constraint / with

- Inline random variables: *write_enable*, *data*
- State variable: *addr*

```
module tb2;
  Packet pkt;
  bit res;

  initial begin
    pkt = new();

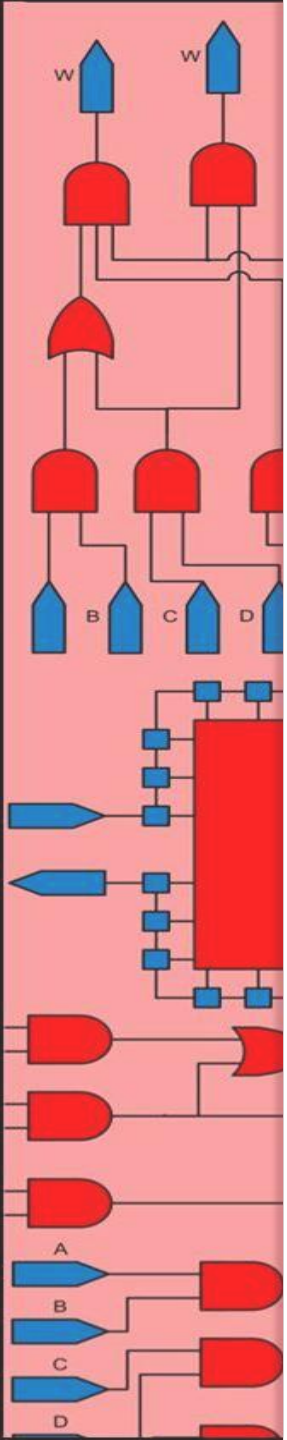
    repeat (10) begin
      if (pkt.randomize()) pkt.display();
      else $display("Randomization failed!");
    end

    res = pkt.randomize() with { addr < 10; data > 200; };
    $display("Randomization %s", res ? "succeeded" : "failed");
    $display("Constrained packet: addr=%0d, data=%0d", pkt.addr, pkt.data);

    repeat (10) begin
      if (pkt.randomize(write_enable, data)) pkt.display();
      else $display("Randomization with argument failed!");
    end

  end
endmodule
```

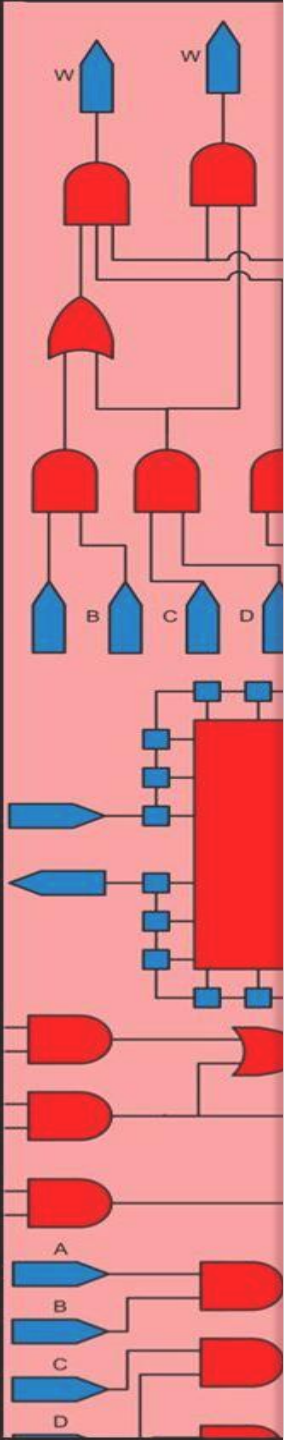
Randomly generates 10 transactions
(a random sequence generator)



Constrained Random Test Generation

Next-level Concepts

- *inside* operator
- Distribution constraints (*dist*)
- Conditional constraints
- Iterative constraints
- Functions in constraints
- *constraint_mode()* method
- *pre_randomize()* and *post_randomize()* methods



Constrained Random Test Generation

Next-level Concepts

- *inside* operator
 - Restrict a random variable to specific ranges or sets of values
 - Cleaner than writing a bunch of || conditions

```
class Packet;  
    rand bit [7:0] opcode;  
  
    constraint valid_opcode {  
        opcode inside { [0:3], [10:15], 25, 30 };  
    }  
endclass
```

opcode can only be:

- Between 0–3
- Between 10–15
- Equal to 25
- Equal to 30

Constrained Random Test Generation

- Example 3

Declaration

```
class Packet;
  rand logic [7:0] addr;
  rand logic [7:0] data;
  rand logic write_enable;

  constraint c_addr_range {addr[1:0] == 2'b00;}
  constraint c_write_enable {
    if (write_enable) addr % 2 == 0;
  }

  function void display();
    $display("ADDR = %0h, DATA = %0h, WRITE_EN = %0b",
             addr, data, write_enable);
  endfunction
endclass

typedef enum {low, mid, high} AddrType;

class ranged_Packet extends Packet;
  rand AddrType atype;
  constraint addr_range {
    (atype == low ) -> addr inside { [0 : 15] };
    (atype == mid ) -> addr inside { [16 : 127] };
    (atype == high) -> addr inside { [128 : 255] };
  }
endclass
```

Like if statement

Supporting OOP features like inheritance, encapsulation, and polymorphism

Usage

```
module tb3;
  ranged_Packet rpkt;
  bit res;

  initial begin
    rpkt = new;

    repeat (5) begin
      if (rpkt.randomize() with {atype == low;}) rpkt.display();
      else $display("Randomization failed!");
    end

  end
endmodule
```

The class *ranged_Packet* inherits all the properties (random variables, constraints, tasks, functions, ...) from the class *packet*

Constrained Random Test Generation

Next-level Concepts

- Distribution constraints (*dist*)
 - Controls **how likely** certain values or ranges are chosen during randomization
 - Example 4

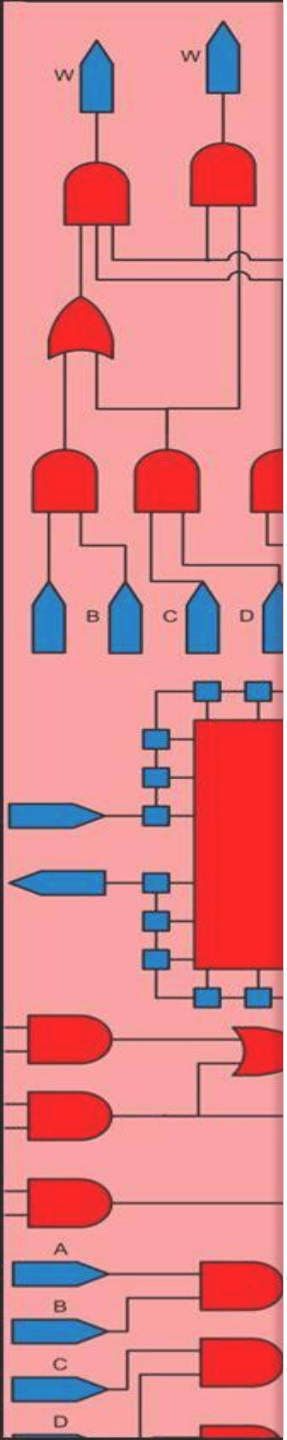
- Range 0–10 gets a 20% chance
- Range 11–100 gets an 80% chance

```
class Packet;
  rand logic [7:0] addr;
  rand logic [7:0] data;
  rand logic write_enable;

  constraint c_addr_range {addr[1:0] == 2'b00;}
  constraint c_addr_distribution {
    addr dist {
      [0:10] := 20,
      [11:100] := 80
    };
  }
  constraint c_write_enable {
    if (write_enable) addr % 2 == 0;
  }

  function void display();
    $display("ADDR = %0h, DATA = %0h, WRITE_EN = %0b",
             addr, data, write_enable);
  endfunction
endclass
```

Declaration



Constrained Random Test Generation

Next-level Concepts

- Conditional constraints
 - Apply rules only when a certain condition is true
 - You can control how randomization behaves based on other random variables
 - Two constructs for declaring conditional relations
 - implications ($\rightarrow$)
If the condition is **false**, then the constraint is ignored (no restriction)
 - *if...else*
Adds an explicit else branch

Constrained Random Test Generation

Next-level Concepts

- Conditional constraints
 - Example
 - “If *pkt_type* is READ, then data must be zero.”

```
constraint c2 {  
    pkt_type == READ -> (data == 0);  
}
```

Using implication

```
constraint c3 {  
    if (pkt_type == READ)  
        data == 0;  
    else  
        data inside {[1:255]};  
}
```

Using if...else

Constrained Random Test Generation

Next-level Concepts

- Iterative constraints
 - Allow arrayed variables to be constrained in a parameterized manner using loop variables and indexing expressions
 - Use the *foreach* or *inside* to iterate over array indices during randomization

specifies iteration over the elements of an array

```
class PacketArray;  
  rand bit [7:0] data[5];  
  
  constraint c_increasing {  
    foreach (data[i]) if (i > 0)  
      data[i] > data[i-1];  
  }  
endclass
```

Declaring a fixed-size array

Constrained Random Test Generation

Next-level Concepts

- Iterative constraints
 - Example 5

Allocate the array with the same size as the burst length

Generate each data word based on its index

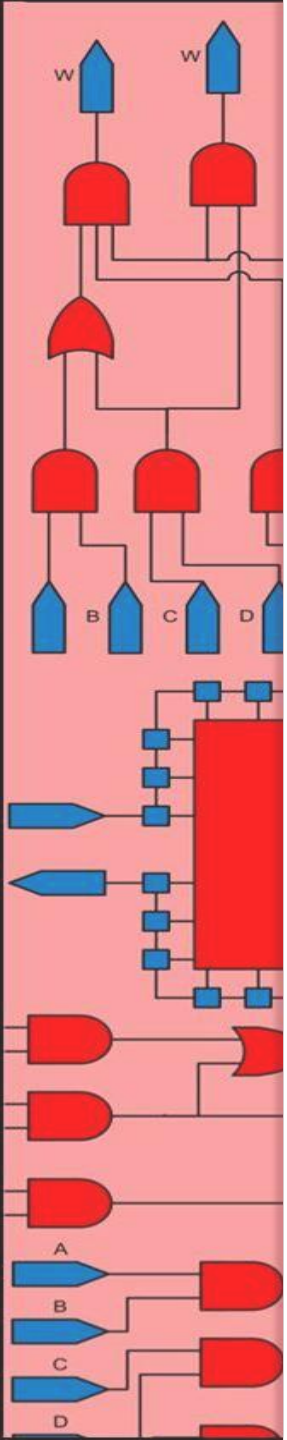
What Happens If the size of a Dynamic Array is not specified?
During `randomize()`, the array has no elements — i.e. its `.size()` is 0.

```
class BurstPacket;
    rand bit [31:0] base_addr;
    rand int unsigned burst_len;
    rand bit [31:0] data[];

    constraint c_burst_len {
        burst_len inside {[2:8]};
    }
    constraint c_array_size {
        data.size() == burst_len;
    }
    constraint c_data_pattern {
        foreach (data[i]) {
            data[i] == base_addr + (i * 4);
        }
    }
    constraint c_addr_align {
        base_addr[1:0] == 2'b00;
    }
    function void display();
        $display("  Base Addr = 0x%08h", base_addr);
        $display("  Burst Len = %0d", burst_len);
        $display("  Data Words:");
        foreach (data[i])
            $display("    [%0d] = 0x%08h", i, data[i]);
    endfunction
endclass
```

Declaration

Declaring a dynamic array



Constrained Random Test Generation

Next-level Concepts

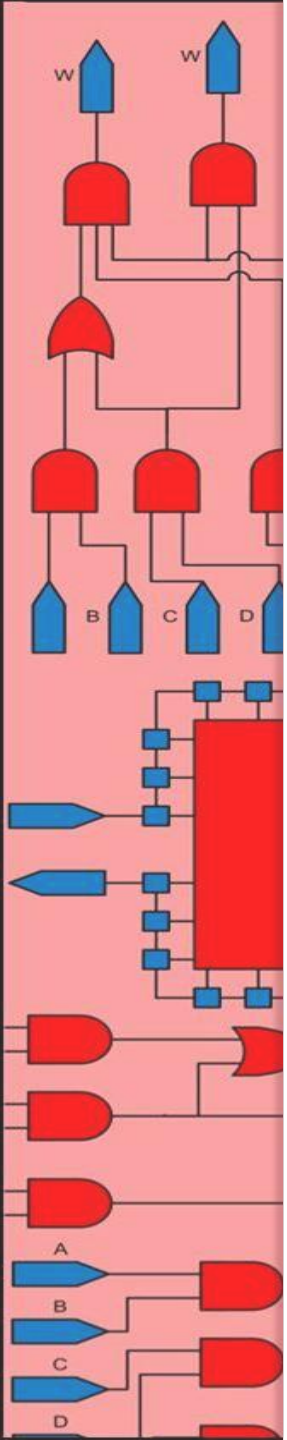
- Functions in constraints
 - Calling a function inside a constraint to express complex mathematical or logical relationships
 - Declare functions in the class variable

Declaration

```
function int count_ones ( bit [9:0] w );  
    for( count_ones = 0; w != 0; w = w >> 1 )  
        count_ones += w & 1'b1;  
endfunction
```

Usage

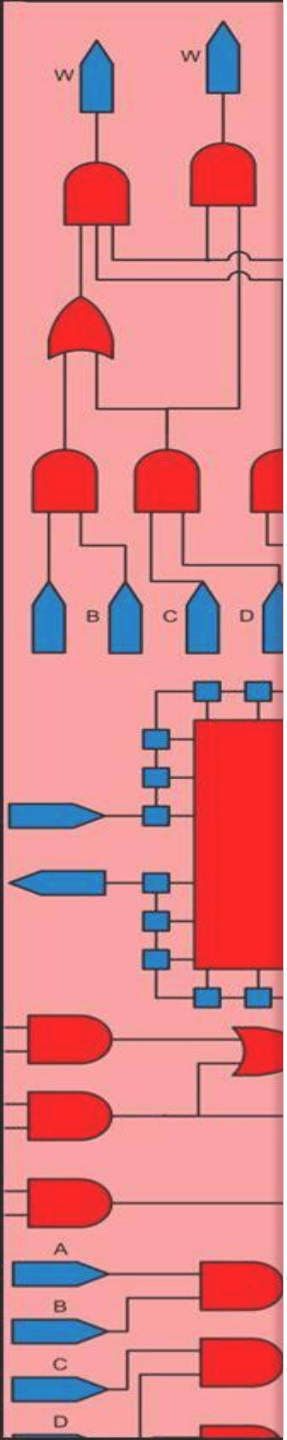
```
constraint C1 { length == count_ones( v ) ; }
```

Constrained Random Test Generation

Next-level Concepts

- *constraint_mode()* method
 - Controls whether a constraint is active or inactive
 - *constraint_mode(0)*: Turns the constraint off
 - *constraint_mode(1)*: Turns the constraint back on
 - *constraint_mode()*: Check current state and returns 1 (on) or 0 (off)



Constrained Random Test Generation

Next-level Concepts

- *pre_randomize()* and *post_randomize()* methods
 - Are special methods inside a class that automatically get called:
 - *pre_randomize()* — right before randomization starts
 - *post_randomize()* — right after randomization completes
 - They let you perform **setup** and **cleanup/checking** tasks during randomization
 - *pre_randomize()* used to
 - Set up variable states
 - Turn constraints on/off
 - Print debug info
 - Adjust dependencies between fields
 - *post_randomize()* used to
 - Check that randomized values are valid
 - Compute derived fields
 - Log results
 - Initialize dependent data

Constrained Random Test Generation

- Example 6

- To occasionally generate “illegal” packets
- Useful for checking how the DUT handles invalid inputs

Declaration

```
class Packet;
  rand logic [7:0] addr;
  rand logic [7:0] data;
  rand logic write_enable;

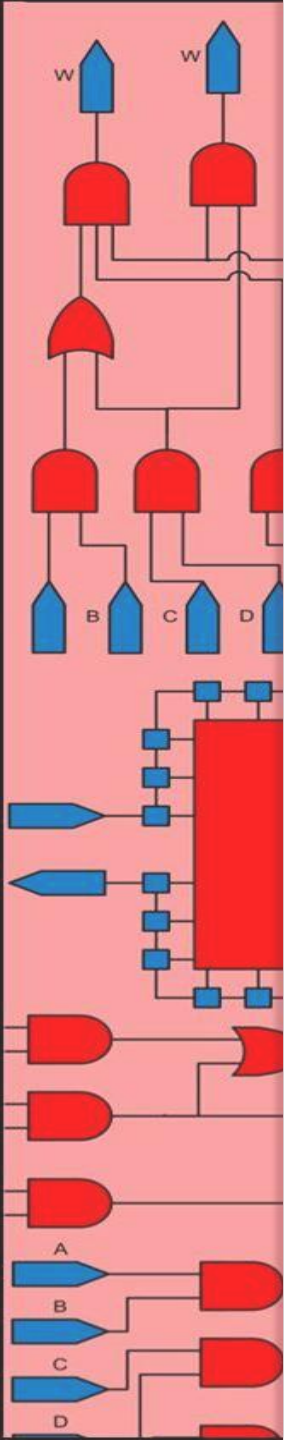
  constraint c_addr_range {
    addr inside {[8'h00 : 8'h7F]};
  }

  constraint c_write_enable {
    if (write_enable)
      addr % 2 == 0;
  }
endclass
```

```
function void pre_randomize();
  $display("\n[pre_randomize] Preparing to randomize packet...");
  if ($urandom_range(0, 3) == 0) begin // 25% of the time
    c_write_enable.constraint_mode(0);
    $display(" [INFO] Disabled constraint 'c_write_enable' for this randomization.");
  end else begin
    c_write_enable.constraint_mode(1);
    $display(" [INFO] Enabled constraint 'c_write_enable'.");
  end
endfunction

function void post_randomize();
  $display("[post_randomize] Randomization done. Checking values...");
  if (!c_write_enable.constraint_mode())
    $display(" [INFO] Constraint 'c_write_enable' was OFF during this run.");
  if (write_enable && addr[0] != 0)
    $display(" Warning: write_enable=1 but addr is odd (constraint off test)");
  $display(" Final values → ADDR=%0d, DATA=%0d, WRITE_EN=%0b", addr, data, write_enable);
endfunction

function void display();
  $display(" ADDR = %0h, DATA = %0h, WRITE_EN = %0b",
    addr, data, write_enable);
endfunction
endclass
```

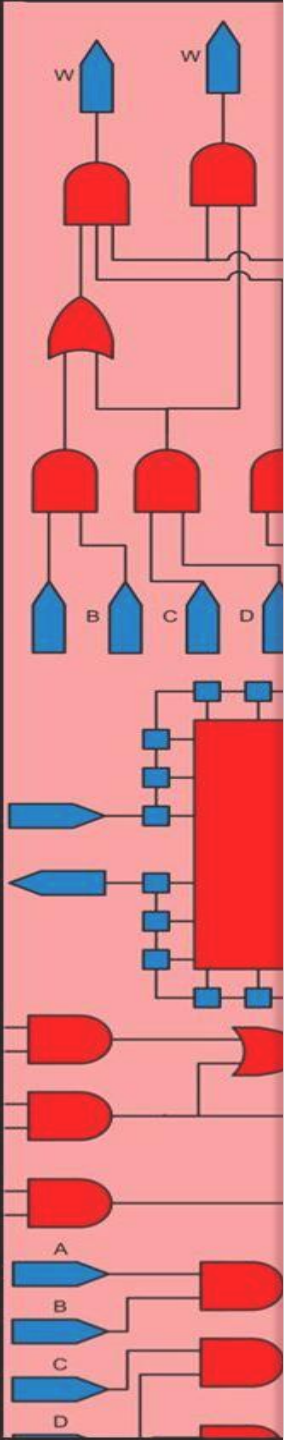


Constrained Random Test Generation

More Concepts

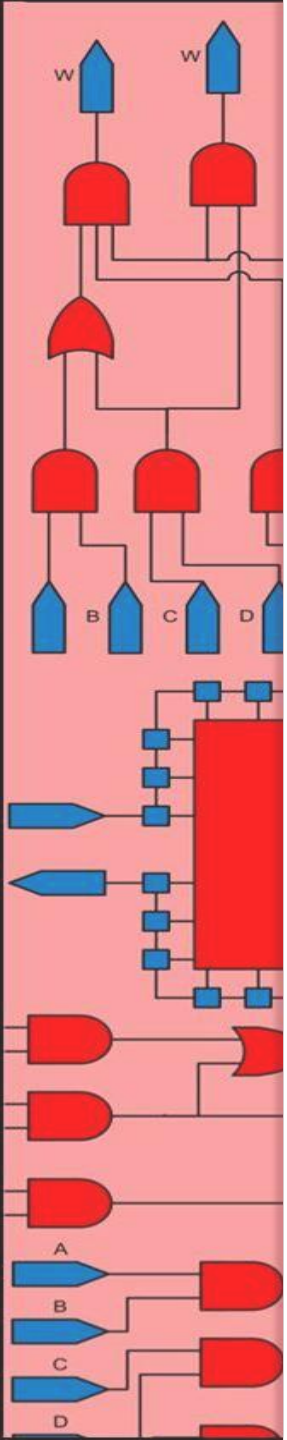
- Variable ordering
- Manually Seeding randomize
- Random weighted case
- ...

Reference: IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800™



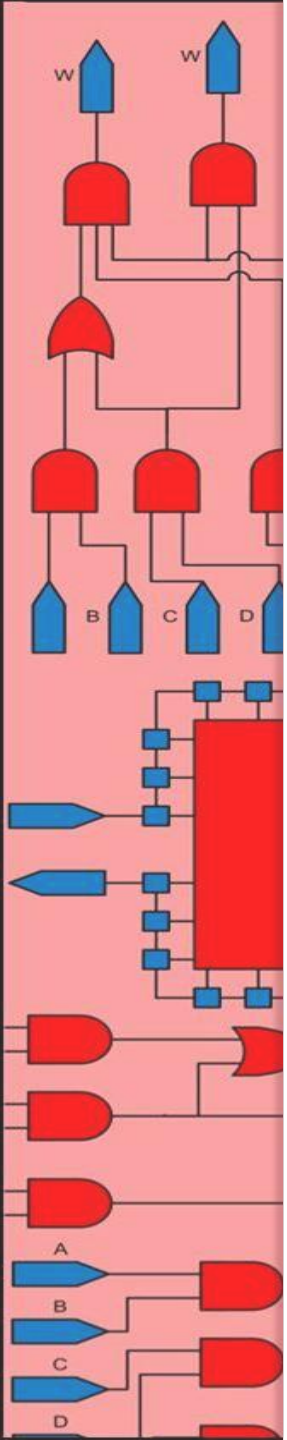
Outline

- Introduction to Digital Verification
- Simulation-based Verification
- SystemVerilog Constructs for Testbenches
- Constrained Random Test Generation
- **SystemVerilog Assertions (SVA)**



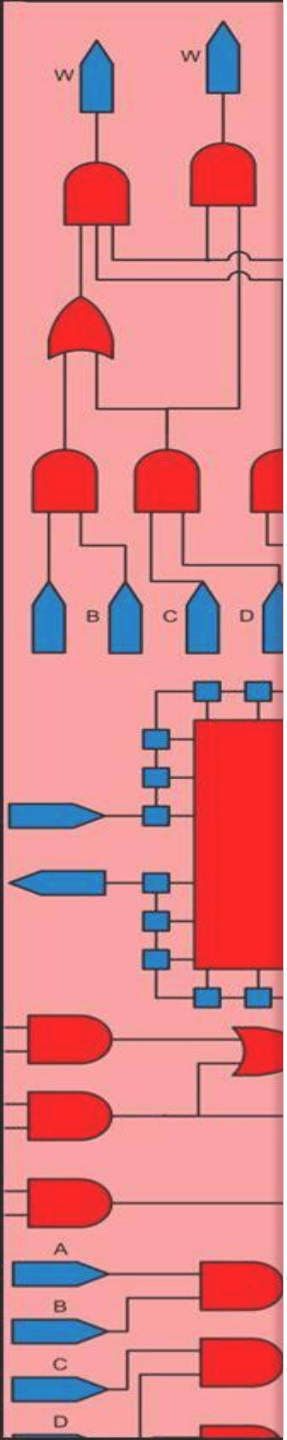
SystemVerilog Assertions (SVA)

- An assertion is a conditional statement that checks for a specific behavior and displays a message if it occurs
- Used to validate design behavior during **simulation** or **formal** verification
- Assertion-based Verification: While the design is being simulated with its testbench data, assertion monitors continuously check for correct design behavior
- In conditions where the design is misbehaving, the monitor is said to fire to alert the designer of the problem
- Assertions are ignored by **synthesis** and can be disabled during simulation



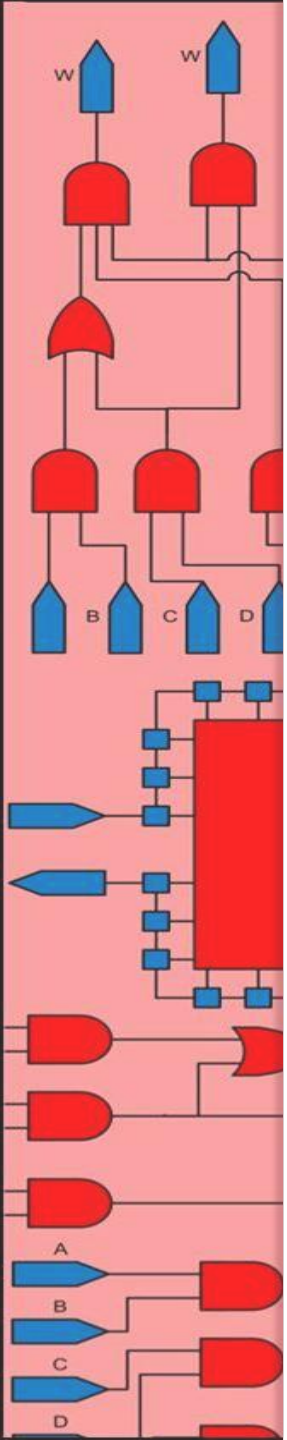
SystemVerilog Assertions (SVA)

- SystemVerilog assertion is a **built-in assertion language** in SystemVerilog, unlike a separate verification language
- Assertion-Based Verification Languages and Standards
 - Property Specification Language (PSL)
 - Open Verification Language (OVL)
 - SystemVerilog Assertions (SVA)
 - SystemC Assertions (SCA)
 - 0-In Assertions (0-In)
 - ...



SystemVerilog Assertions (SVA)

- Two kinds of SystemVerilog Assertions: Immediate & Concurrent
- Immediate Assertions
 - Check statements in the procedural blocks
 - No timing
 - Used inside procedural blocks, like always, initial, task, function
- Concurrent Assertions
 - Are temporal domain assertions
 - Execute in parallel with other always blocks concurrently
 - Work on the clock edge (sampling edge) → edge sensitive
 - Allow to check the sequence of events spread over multiple clock cycles
 - Can be used in procedural blocks, modules, programs



SystemVerilog Assertions (SVA)

Immediate Assertions

- Syntax:
- An immediate assertion may include a *pass* block and/or a *fail* block
- The expression is non-temporal
- Interpreted as an expression in an *if* statement
 - Evaluates to 1 → pass
 - Evaluates to 0, X, Z → fail

```
assert label: assert (expression)
    pass block code
else
    fail block code
```

SystemVerilog Assertions (SVA)

Immediate Assertions

- Example

```
module BCD_Counter (  
    input logic rst, clk,  
    output logic [3:0] cnt);  
  
    always_ff @(posedge clk) begin  
        if (rst || cnt >= 9) cnt <= 0;  
        else cnt <= cnt + 1;  
  
        bcd_assert : assert ((cnt >= 0) && (cnt <= 9)) begin  
            $display ("Correct count range");  
        end else begin  
            $display ("Error: count out of range at time %0t, cnt = %0d", $time, cnt);  
        end  
  
    end  
endmodule
```

Assertion Label

Assertion Expression

Pass Statement

Fail Statement

```
# Error: count out of range at time 697, cnt = x  
# Error: count out of range at time 731, cnt = x  
# Error: count out of range at time 765, cnt = x  
# Error: count out of range at time 799, cnt = x  
# Error: count out of range at time 833, cnt = x  
# Correct count range  
# Correct count range  
# Correct count range
```

SystemVerilog Assertions (SVA)

Immediate Assertions

- The fail block of an assertion has a **severity** associated with it
- By default, the severity of an assertion failure is *error*

- The *finish_number* tells the simulator how to end the simulation
- Optional field, 1 by default.

\$fatal

\$fatal [(**finish_number**, “message”, message arguments)]

\$error

\$error [(“message”, message arguments)]

\$warning

\$warning [(“message”, message arguments)]

\$info

\$info [(“message”, message arguments)]

SystemVerilog Assertions (SVA)

Immediate Assertions

- Example

```
module BCD_Counter (
    input logic rst, clk,
    output logic [3:0] cnt);

    always_ff @(posedge clk) begin
        if (rst || cnt >= 9) cnt <= 0;
        else cnt <= cnt + 1;

        bcd_assert : assert ((cnt >= 0) && (cnt <= 9)) begin
            $display ("Correct count range");
        end else begin
            $fatal (2, "Error: count out of range at time %0t, cnt = %0d", $time, cnt);
        end
    end

end
endmodule
```

finish number is 0, 1 and 2

0: Terminate gracefully — same as *\$finish*

1: Terminate immediately — aborts simulation

2: Terminate abruptly — like a crash

Terminates execution of the simulation

```
# doing the normal procedure
# ** Fatal: two grant signals are active at the same time at 0
# Time: 17 ps Scope: arbiter_Tester.UUT File: C:/Users/Sheida/Desktop/WPI- S2019/SVA_examples/arbiter.sv Line: 8
# ** Note: Data structure takes 9126480 bytes of memory
# Process time 0.25 seconds
# $finish : C:/Users/Sheida/Desktop/WPI- S2019/SVA_examples/arbiter.sv(8)
# Time: 17 ps Iteration: 1 Instance: /arbiter_Tester/UUT
```


System Verilog Assertions

Concurrent Assertions

- ▶ Allow to check a sequence or behavior spread over time, over multiple clock cycles
- ▶ A sequence is a series of true/false expressions spread over one or more clock cycles
- ▶ Execute as a background process in parallel with RTL code

Concurrent Assertions examples

A *Req.* should be **followed** by an *Ack.* occurring **no more than** two clocks after the *Req.* is asserted

```
req_ack: assert property (@(posedge clk) req | -> ##2 ack)  
         else $error;
```

If *req* is true, then *ack* must be true 2 clock cycles later (on the 2nd clock after *req*)

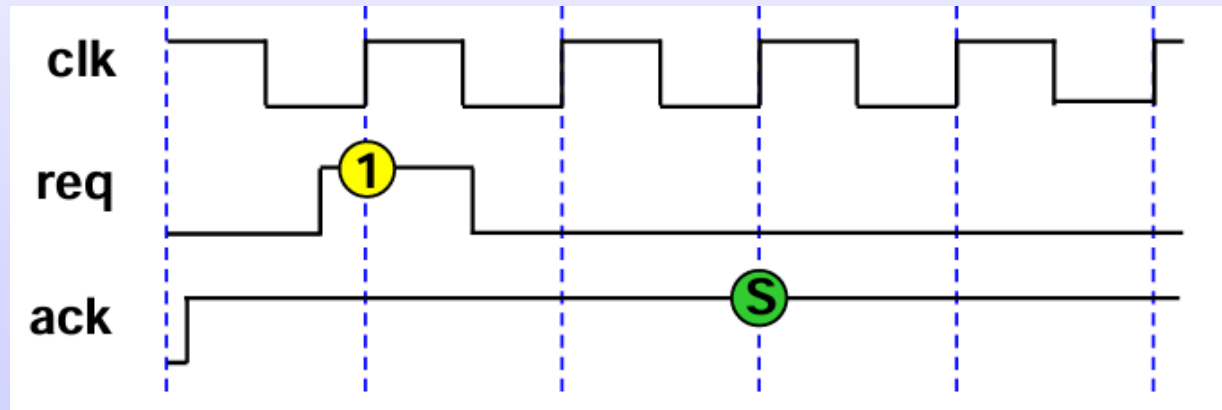
System Verilog Assertions

55

Concurrent Assertions

- Concurrent assertions are evaluated only at the occurrence of a *clock tick*

```
req_ack: assert property (@(posedge clk) req | -> ##2 ack)  
else $error;
```



If *req* is true, then *ack* must be true 2 clock cycles later (on the 2nd clock after *req*)

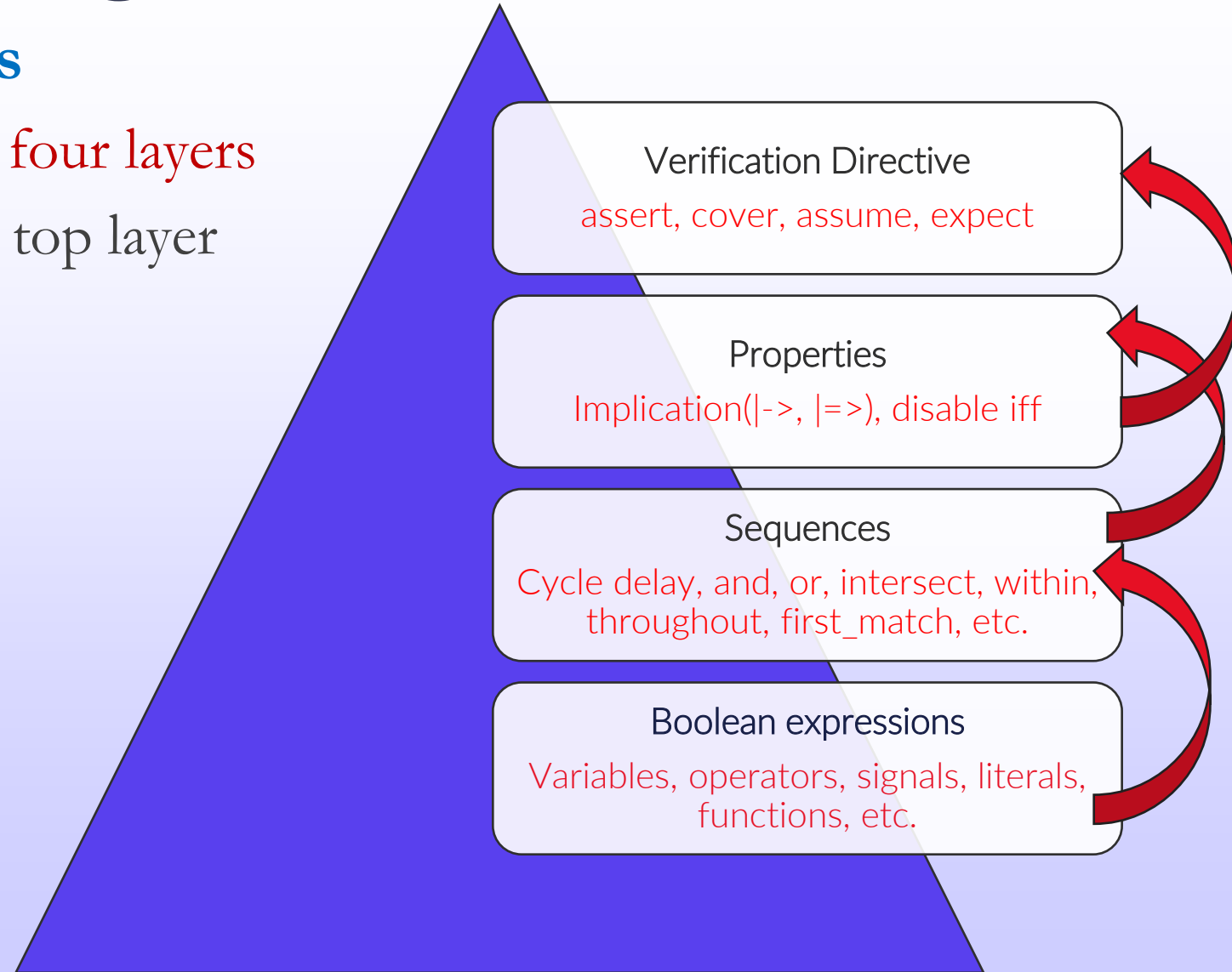
System Verilog Assertions

56

Concurrent Assertions

Concurrent assertions has **four layers**

Each layer is used to build top layer



System Verilog Assertions

An example to show each layer

```
module arbiter_concurrent_assert (input logic clk, req, output logic grant);
```

```
sequence req_grant_seq;
  (~req & grant) ##1 (~req & ~grant);
endsequence
```

Boolean layer

Sequence layer

```
property req_grant_prop;
  @ (posedge clk)
    req | => req_grant_seq;
endproperty
```

Property layer

```
req_grant_assert : assert property (req_grant_prop)
else $display("@%0dns Assertion Failed", $time);
```

Assertion
directive layer

```
always_ff @ (posedge clk)
  grant <= req;
```

```
endmodule
```

System Verilog Assertions

Boolean layer

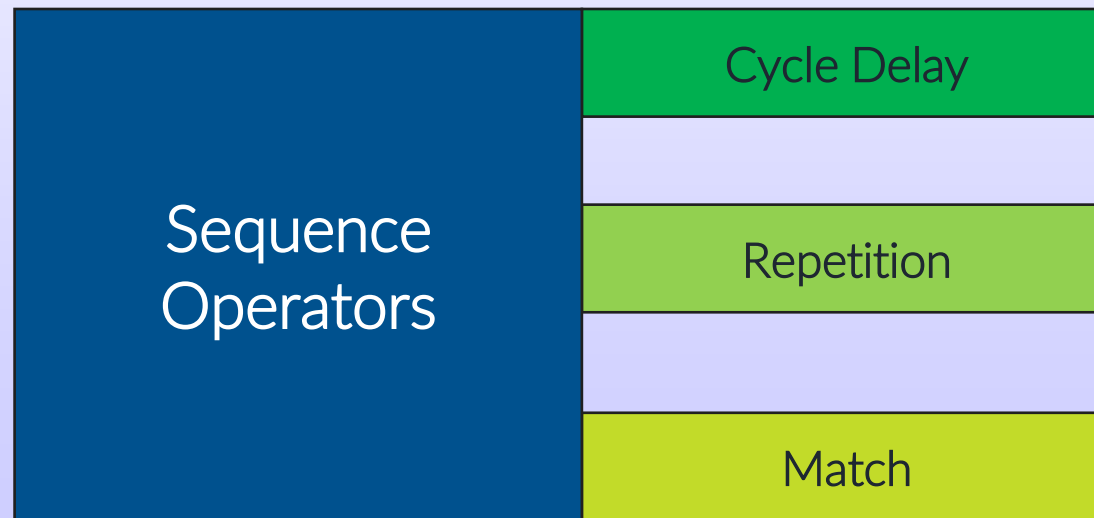
- Lowest layer in concurrent assertions
- The result is true or false
- Checking is done on variables
- Functions can also be used

```
(~req & grant) ##1 (~req & ~grant);
```

System Verilog Assertions

Sequence layer

- Uses the Boolean layer to construct valid sequence of events.
- Different operators are used for building the sequences



System Verilog Assertions

60

Cycle Delay Operators

n

Fixed number of n cycles

means delay

```
req_ack: assert property (@(posedge clk) req |-> ##3 ack)  
else $display("error at %t:", $time);
```

[min : max]

A range of clock cycles

```
req_ack: assert property (@(posedge clk) req |-> ##[1:3] ack)  
else $display("error at %t:", $time);
```

ack must be true
between 1 and 3
clocks later

System Verilog Assertions

Repetition Operators

[*

Consecutive repetition

[->

Goto repetition

[=

Nonconsecutive repetition

System Verilog Assertions

62

Repetition Operators

[*

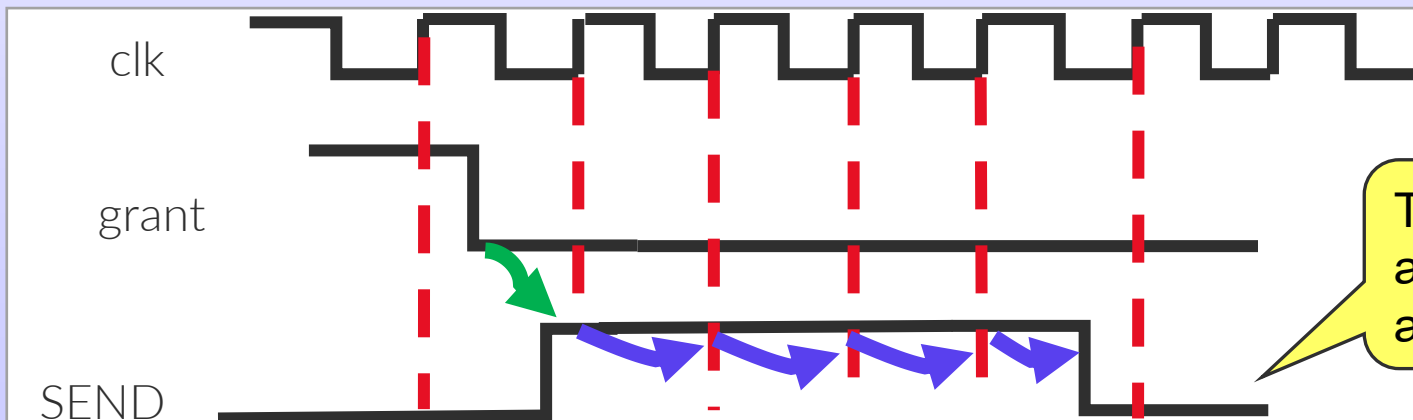
Consecutive repetition

specifies finitely many iterative matches of the operand sequence, with a delay of one clock tick from the end of one match to the beginning of the next

```
sequence Send_Period;  
  (@posedge clk) ~grant |-> ##1 SEND[*4] ##1 ~SEND
```

```
endsequence  
sequence Send_Period;  
  (@posedge clk) ~grant |-> ##1 SEND ##1 SEND ##1 SEND ##1 SEND ##1 ~SEND  
endsequence
```

Same as:



The *SEND* signal must be active for 4 machine cycles and then should drop to zero

System Verilog Assertions

63

Repetition Operators

[*

Consecutive repetition

```
sequence Busy_length;  
  (@posedge clk) Busy[*1:4] |-> ##1 !Busy  
endsequence
```

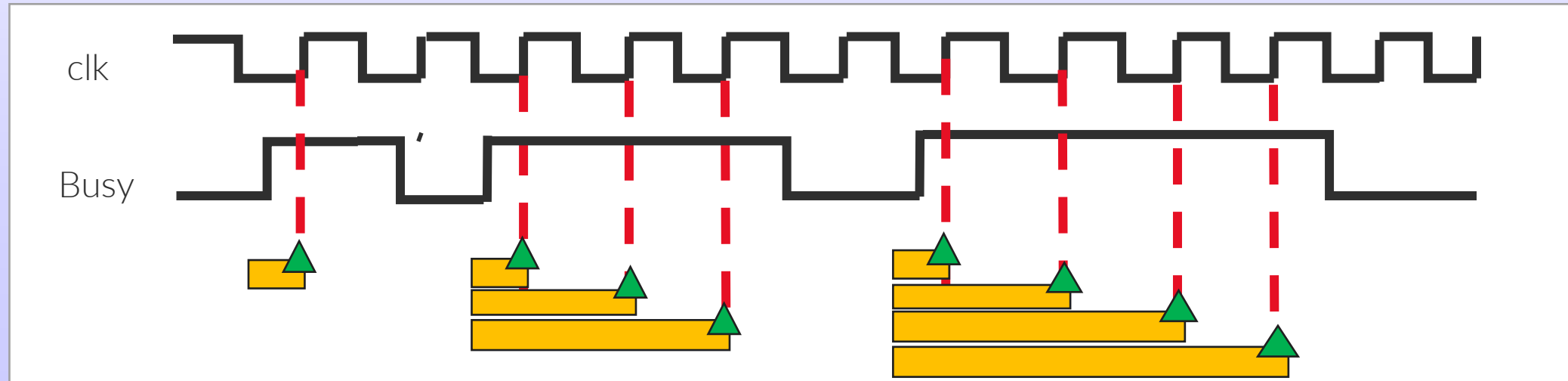
Minimum and maximum of the *busy* period are 1 and 4 clock cycles

Busy

Or (Busy ##1 Busy)

Or (Busy ##1 Busy ##1 Busy)

Or (Busy ##1 Busy ##1 Busy ##1 Busy)



System Verilog Assertions

64

Repetition Operators

[->

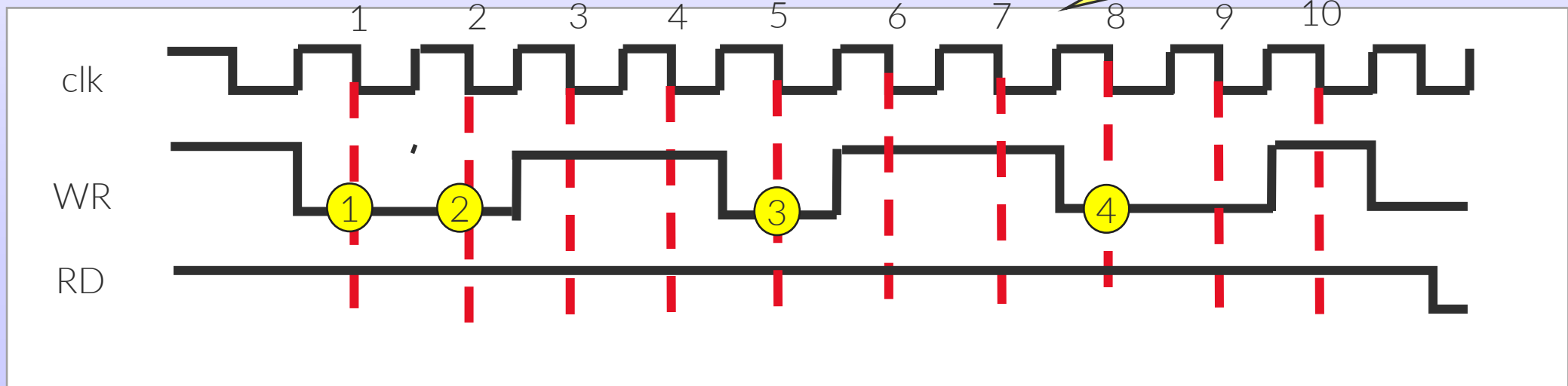
Goto repetition

Specifies a **non-consecutive** sequence; once and forever

The sequence with the *goto repetition* must always complete on an expression match

```
sequence Write_4_Goto;  
  @(negedge clk) (!WR & RD) [->4];  
endsequence
```

Starting from clock 1, then it will complete on clock 8



System Verilog Assertions

65

Repetition Operators

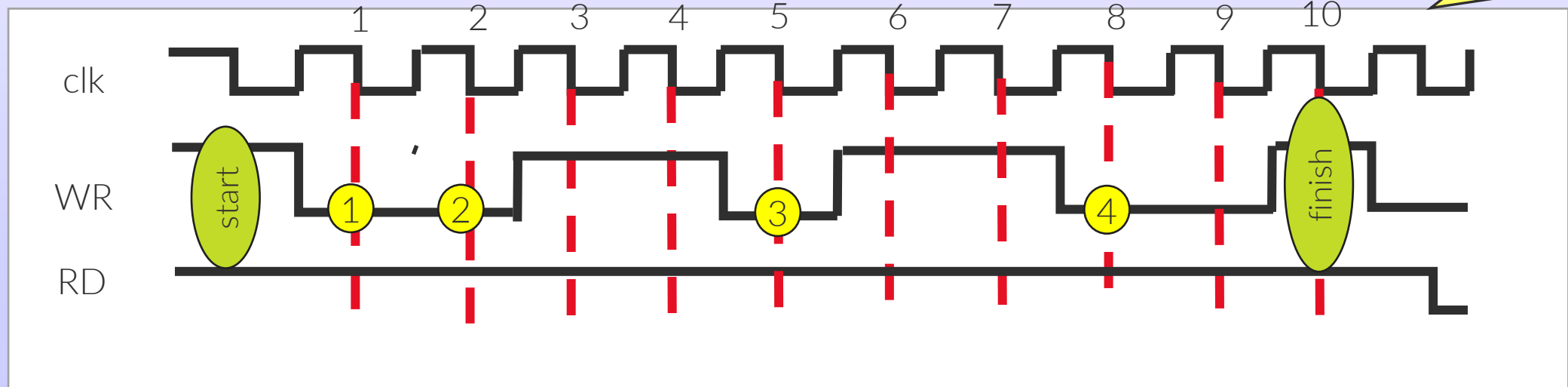
[= Nonconsecutive repetition

Specifies a **non-consecutive** sequence

Repetitions can occur non-consecutively over time

```
sequence Write_4;  
  @(negedge clk) (!WR & RD) [=4];  
endsequence
```

can also start or complete at a clock cycle, in which expression (!WR & RD) is false



System Verilog Assertions

Sequence Operators

Operator	Description
##m	Clock delay
##[m:n]	
[*m]	Repetition—consecutive
[*m:n]	
[=m]	Repetition—non consecutive
[=m:n]	
[->m]	GoTo repetition—non consecutive
[->m:n]	
sig1 throughout seq1	Signal sig1 must be true throughout the sequence seq1
seq1 within seq2	Sequence seq1 must be contained within sequence s2
seq1 intersect seq2	‘intersect’ of two sequences; same as ‘and’ but both sequences must also ‘end’ at the same time
seq1 and seq2	‘and’ of two sequences. Both sequences must start at the same time but may end at different times
seq1 or seq2	‘or’ of two sequences. It succeeds if either sequence succeeds
first_match complex_seq1	Matches only the first of possibly multiple matches

System Verilog Assertions

Property Layer

Properties can be declared inside:

- ➡ A module
- ➡ A program
- ➡ A Procedural Block

System Verilog Assertions

Property Layer

A sequence can be conditioned with an **implication operator**

- If the condition is true, the sequence is evaluated
- If the condition is false, the sequence is not evaluated (a don't care)

| -> **overlapped** implication: sequence evaluation starts **immediately**

| => **non-overlapped** implication: sequence evaluation starts **at the next clock**

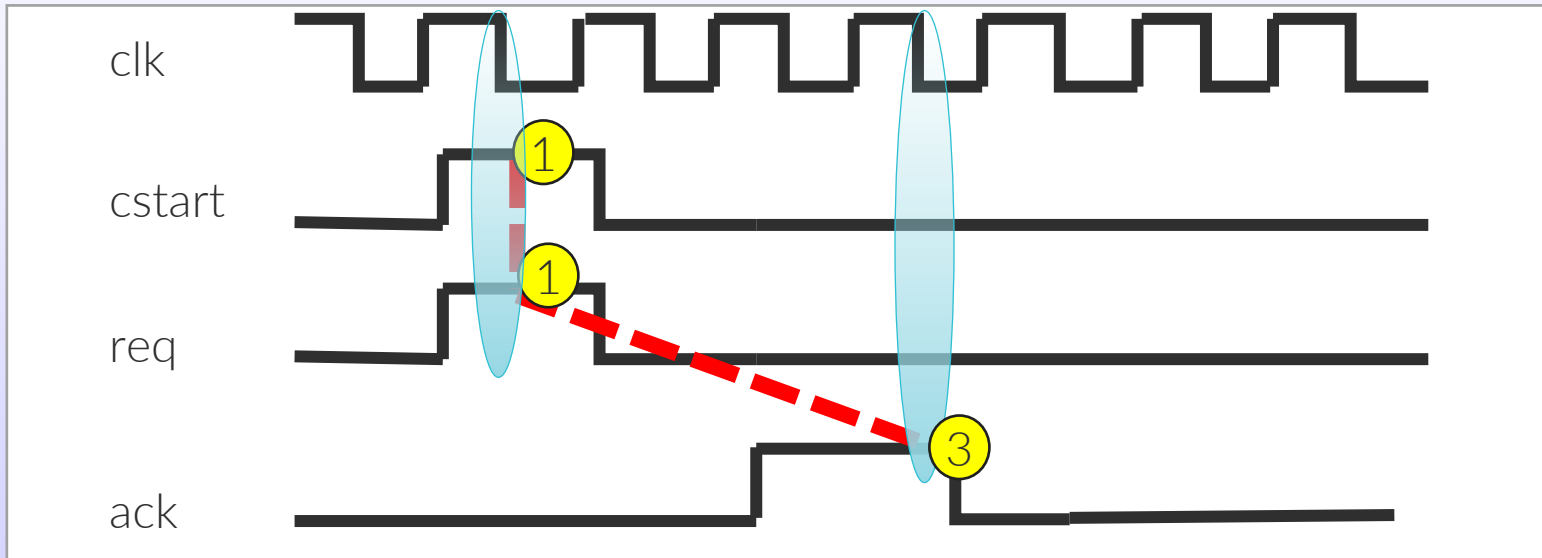
System Verilog Assertions

69

Property Types:

Overlapped Implication

```
req_ack: assert property (@(negedge clk) cstart |-> (req ##3 ack))
```



if *cstart* is true at the **negedge of the clock**, it implies that *req* must be true **at the same negedge clock** and the *ack* must be true **3 clock cycles later**

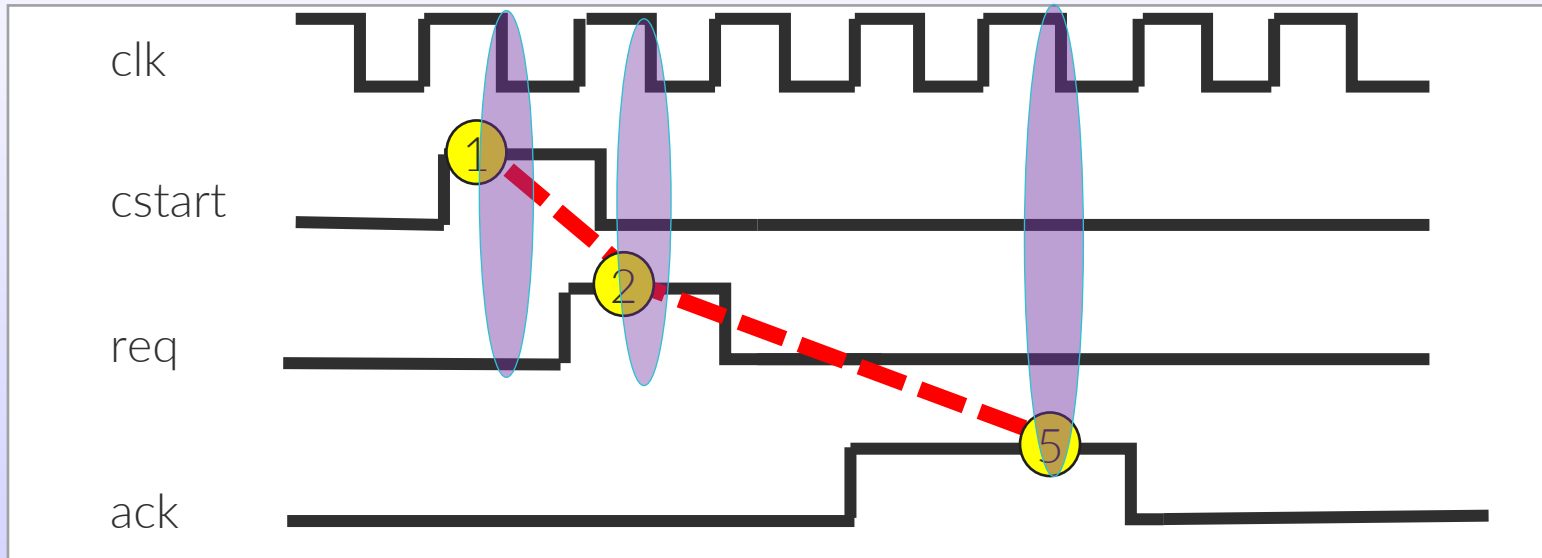
System Verilog Assertions

70

Property Types:

Nonverlapped Implication

```
req_ack: assert property (@(negedge clk) cstart ==> (req ##3 ack))
```



if *cstart* is true at the **negedge of the clock**, it implies that *req* must be true **at the next negedge clock** and the *ack* must be true **3 clock cycles later**

System Verilog Assertions

Property Types

- A Sequence
- Negation
- Disjunction
- Conjunction
- If ... else
- Implication
- Disable if

System Verilog Assertions

Property Types:

- **Sequence**: A simple sequence expression

```
property fifo_should_go_empty;  
    (cnt==0 && read && !write ==> empty);  
endproperty
```

- **Negation**: A negation property is same as logic not operator on outcome of a property

```
property fifo_should_go_empty;  
    not (cnt==0 && read && !write ==> empty);  
endproperty
```

System Verilog Assertions

Property Types:

➤ disable iff

- 'disable iff' operator allows you to express a property under a given condition.

Disable assertion
testing if reset is low

```
property empty_fifo_condition;  
  @(posedge clk) disable iff (!rst_n) (cnt>0 |-> !empty);
```


System Verilog Assertions

Property Types:

► Disjunction

- Like a logical *or* operator, where at least one of the properties evaluates to **true**, then

Disjunction property evaluates to **true**

```
property fifo_should_go_empty;  
  @(posedge clk) disable iff (!rst_n)  
    (cnt==0 && read && !write | => empty);  
endproperty
```

```
property empty_read_empty;  
  @(posedge clk) disable iff (!rst_n)  
    (empty && read && !write | => empty);  
endproperty
```

```
property fifo_Disjunction;  
  @(posedge clk) (fifo_should_go_empty) or (empty_read_empty);  
endproperty
```

System Verilog Assertions

Property Types: Sequence

► Conjunction

- Like a logical *and* operator, where both properties evaluate to **true**, then Conjunction property evaluates to **true**

```
property empty_fifo_condition;  
  @(posedge clk) disable iff (!rst_n)  
    (cnt>0 |-> !empty);  
endproperty  
  
property read_rptr_change;  
  @(posedge clk) disable iff (!rst_n)  
    (!empty && read && !write |=> (rptr_inc));  
endproperty  
  
property fifo_Conjunction;  
  @(posedge clk) (empty_fifo_condition) and (read_rptr_change);  
endproperty
```

System Verilog Assertions

Property Types:

► if .. else

- if a certain condition is **true**, one property is taken, otherwise the property becomes what is in the **else** part

```
property empty_fifo_condition;  
  @(posedge clk) disable iff (!rst_n)  
    (cnt>0 |-> !empty);  
endproperty  
  
property read_rptr_change;  
  @(posedge clk) disable iff (!rst_n)  
    (!empty && read && !write |=> (rptr_inc));  
endproperty  
  
property fifo_ifelse;  
  @(posedge clk) if (check) empty_fifo_condition else read_rptr_change;  
endproperty
```

System Verilog Assertions

Verification Directives

assert

To specify that the given property of the design is true in simulation

assume

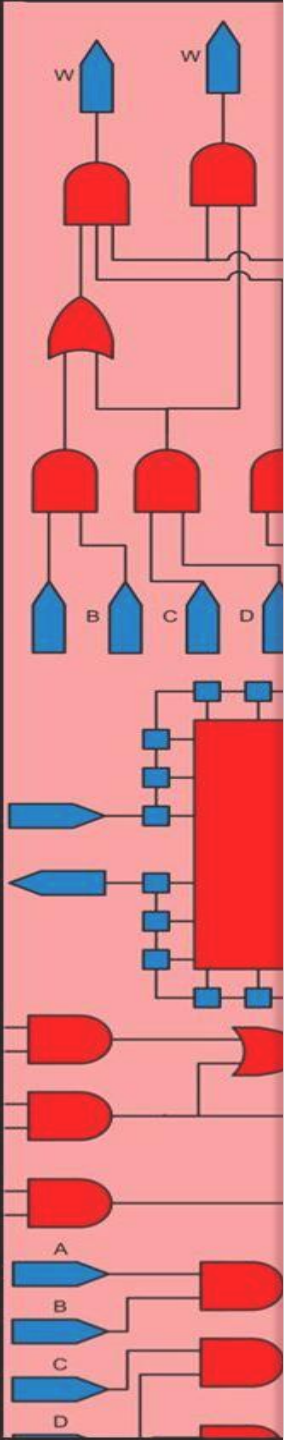
To specify that the given property is an assumption, used by formal tools to generate input stimulus

cover

To evaluate the property for functional coverage

restrict

To specify the property as a constraint on formal verification computations which is ignored by simulators



Conclusion

An overview of

- Digital Verification Methods
- Simulation-based Verification
- SystemVerilog Constructs for Testbenches
- Constrained Random Test Generation
- SystemVerilog Assertions (SVA)